

How To Use This Guide

Think of this guide like a recipe book, except instead of cooking food, we are cooking up a game! Each week is a new step in the recipe, and every step builds on the one before it. Don't skip ahead Week 4's game mechanics only make sense once you know the ingredients from Weeks 1 through 3.

Here is what you will find inside every lesson:

- A short explanation of the new idea, using everyday comparisons.
- A piece of real Python code that shows the idea in action.
- A box called "Here's What's Happening" that walks through the code one line at a time, in plain English.
- A "Mini Challenge" so you can practice the idea yourself before moving on.
- Occasional "Tip" boxes with shortcuts, warnings, or fun facts.

□ Here's What's Happening:

Whenever you see this box, it means we are about to slow down and look at code super closely.

We will never just show you code and expect you to guess what it does every single line gets explained, the same way a patient friend would explain it to you while sitting next to you at the computer.

□ Tip:

Keep a notebook (real paper or digital) next to you as you go through this guide. Jot down new words like variable, function, or loop. You'll be surprised how quickly they become part of your everyday vocabulary!

The Big Goal

Every single lesson in this guide points toward one destination: a finished, playable game that you built with your own hands and your own code. Here is the roadmap:

1. Week 1 Set up your tools and learn the basic building blocks of Python: variables, data types, and simple input/output.
2. Week 2 Teach your programs to make decisions and repeat actions, using conditionals, loops, and randomness.
3. Week 3 Organize your code into reusable functions, store game data in lists and dictionaries, and draw your first graphics with the turtle module.

4. Week 4 Combine everything into a real game loop with keyboard controls, collisions, scoring, and win/lose conditions.
5. Week 5 Polish your game, hunt down bugs like a professional, and get ready to show it off to family and friends.

Ready? Grab your computer, take a deep breath, and let's start coding!

WEEK 1

Python Foundations & Setup

Building your workshop and learning the first tools

Welcome to Programming!

Welcome! You are about to start one of the most exciting journeys you can take on a computer: learning to speak its language. Over the next five weeks, you will go from typing your very first line of code to building a complete, playable video game.

You don't need any experience to start. You don't need to be a math genius, and you don't need a fancy computer. You just need curiosity, a little patience, and a willingness to try things, make mistakes, and try again.

What is Programming?

Programming is the process of giving a computer a clear set of instructions to perform specific tasks and solve problems. It allows us to create things that didn't exist before games, mobile apps, websites, robots, and even artificial intelligence. Every app on your phone, every game you've played, and every website you've visited began as a collection of instructions written by a programmer.

Programmers are the creative minds behind almost everything you interact with on a screen. They carefully think through problems such as: *How should the character move? What should happen when a player presses a button? How does the game know when the player has won?* They then write step-by-step instructions that tell the computer exactly how to make those things happen.

💡 Tip:

Mistakes are not failures in programming they are opportunities to learn. Professional programmers make mistakes every day. The difference between a beginner and an expert isn't that experts never make mistakes; it's that they have more experience finding, understanding, and fixing them.

What You'll Need

- A computer (Windows, macOS, or Linux all work fine) with an internet connection for downloading tools.
- About 45–60 minutes of quiet, focused time for each lesson.
- A notebook or blank document for jotting down new vocabulary as you meet it.

What Is Coding?

Coding is the process of writing instructions that tell a computer what to do using a programming language such as **Python**. It is the act of turning ideas into code that a computer can read and execute.

However, **coding is only one part of programming**.

Programming is the complete process of creating a computer program. It involves several important steps, including:

1. **Planning** – Thinking about the problem and deciding what the program should do.
2. **Designing** – Organizing how the program will work and how different parts will fit together.
3. **Coding** – Writing the instructions using a programming language.
4. **Testing** – Running the program to check whether it works correctly.
5. **Debugging** – Finding and fixing mistakes (called *bugs*).
6. **Improving** – Adding new features and making the program better over time.

The programming language we will use throughout this course is **Python**. Python is one of the most popular programming languages in the world because its syntax is simple, readable, and similar to everyday English. This makes it an excellent language for beginners while also being powerful enough for professional programmers who build games, websites, artificial intelligence, and many other real-world applications.

💡 Remember:

Programming is the complete journey of creating a computer program, while **coding** is the important step where we write the instructions that bring our ideas to life. Every programmer is a coder, but good programming also requires planning, testing, debugging, and improving the program.

Algorithms: The Plan Behind the Code

Before you even write a single line of Python, it helps to plan out your steps in plain English first. This plan is called an algorithm a clear sequence of steps that solves a problem or completes a task, whether or not a computer is involved.

For example, here's an algorithm for deciding whether a player has won a game:

6. Check the player's score.
7. If the score is 500 or higher, the player has won.
8. Otherwise, the game continues.

Notice this algorithm doesn't use any Python syntax at all it's just a clear, ordered plan. Once you have a solid algorithm, translating it into Python becomes much easier, because you already know exactly what needs to happen.

□ **Tip:**

Whenever a coding problem feels overwhelming, stop typing and write out your algorithm in plain English or on paper first. Professional programmers do this constantly it's often called "pseudocode."

What Is Python, and Why Python?

Python is a programming language created in the late 1980s by a programmer named Guido van Rossum. It was named after the British comedy show Monty Python's Flying Circus not the snake! Here's why Python is a fantastic first language:

- Readable: Python code looks a lot like English sentences, so it's easier to read and understand than many other languages.
- Popular: Python powers everything from Instagram's backend to NASA's data science tools to the games you'll build in this guide.
- Beginner-friendly: Python doesn't require a lot of extra symbols and punctuation just to get started.
- Huge community: Millions of people use Python, which means if you ever get stuck, chances are someone has already asked (and answered) your exact question online.

Compare a simple instruction in Python to the same instruction in some other popular languages. In several other languages, printing a message requires extra punctuation like semicolons and curly braces. In Python, it's simply:

```
print("Hello, World!")
```

□ Here's What's Happening:

There's no code to break down line-by-line yet this section is just background. But keep this in mind: every single piece of code you write from here on will be written in Python, so understanding why it exists will help the syntax make more sense later.

What Can You Build With Python?

By the time you finish this guide, you'll have built a game. But Python's uses stretch far beyond games:

- Websites and web applications (using tools like Django or Flask).
- Data analysis and science Python is one of the most popular languages for working with data.
- Automation writing small scripts that handle boring, repetitive tasks automatically.
- Artificial intelligence and machine learning research.
- Desktop applications, robotics, and yes plenty of games!

Setting Up Your Workshop

Every builder needs a workshop. A carpenter needs a workbench and tools; a painter needs a canvas and brushes. As a programmer, your workshop is made of two things: Python itself (the language) and an editor called PyCharm (the tool you'll use to write and run your code).

Installation Steps

Follow these steps carefully. Ask an adult for help if you run into any permission pop-ups on your computer that's completely normal.

9. Open your web browser and go to python.org.
10. Click the Downloads tab and choose the version for your operating system (Windows, macOS, or Linux).
11. Run the downloaded installer. On the very first screen, check the box that says "Add Python to PATH" this is important, so don't skip it!
12. Click Install Now and wait for the installation to finish.
13. Next, go to jetbrains.com/pycharm and download the free Community Edition.
14. Run the PyCharm installer, accepting the default options, and let it finish.

15. Open PyCharm for the first time and choose a color theme Light or Dark whichever is easier on your eyes.

□ **Tip:**

"Add Python to PATH" tells your computer where to find Python whenever you ask for it. If you forget to check this box, don't worry you can always re-run the installer and check it later.

Once both are installed, verify everything worked correctly:

16. Open PyCharm.
17. Click "New Project."
18. Look for a dropdown or label that mentions a Python interpreter it should list the version you just installed, like Python 3.12.
19. If PyCharm can't find any Python interpreter, close it, make sure Python finished installing correctly, and reopen PyCharm.

Understanding the PyCharm Interface

When you open PyCharm and create a new project, you'll see a few important areas on the screen:

- Project panel (left side): shows all the files in your project, like a filing cabinet for your code.
- Editor (middle, biggest area): this is where you actually type your Python code.
- Run window (bottom): this is where your program's output appears when you run it, and where error messages show up if something goes wrong.
- Run button (green triangle, top right): click this whenever you want your code to spring to life.

□ **Here's What's Happening:**

Think of the Editor as your notebook paper, the Run window as the stage where your code performs, and the green Run button as the light switch that turns everything on.

Every project you build in this guide including your final game will live inside a PyCharm project, so getting comfortable with this layout now will save you time later.

A few more helpful interface details worth knowing:

- Line numbers appear on the left side of the Editor error messages will often reference these numbers, so you can jump straight to the problem.
- Red squiggly underlines usually mean a typo or syntax mistake hover your mouse over one to see PyCharm's suggestion.

- The Stop button (a red square, next to the Run button) forces a running program to quit useful if you accidentally create an infinite loop, which you'll learn about in Week 2.

□ **Mini Challenge: Set Up Your First Project**

Open PyCharm and create a new project called PythonGameJourney.

Inside that project, create a new Python file named week1.py.

Leave it empty for now in the next section, we'll write our very first line of code together!

Your First Program: Hello, World!

It's a tradition almost as old as programming itself: the very first program you write in any new language is one that displays the words "Hello, World!" on the screen. Let's carry on that tradition.

Type this into your week1.py file:

```
print ("Hello, World!")
```

Now click the green Run button. You should see the words Hello, World! appear in the Run window at the bottom of your screen. Congratulations you just wrote and ran your first computer program!

Breaking Down the print() Function

That one line of code has several important parts. Let's slow down and look at each piece.

```
print("Hello, World!")
```

□ **Here's What's Happening:**

print is the name of a built-in Python function a ready-made command that Python already understands. Its job is simple: display whatever you give it onto the screen.

The parentheses () come right after print. They are how we hand information over to the function. Whatever goes inside the parentheses is what print will show.

"Hello, World!" is the piece of information we handed to print. The quotation marks tell Python, "everything between these two marks is text, not code just display it exactly as written." In programming, a piece of text like this is called a string.

When you clicked Run, Python read this single line from left to right, saw the print function, opened the parentheses, found the string "Hello, World!", and displayed it in the Run window.

□ **Tip:**

You can print anything you want between the quotation marks. Try changing the message to your own name, like `print("Hello, my name is Ada!")` and run it again to see the change.

Printing Multiple Things at Once

`print()` can actually accept more than one piece of information at a time, separated by commas.

```
print("Score:", 100)
```

□ **Here's What's Happening:**

Here we've given `print()` two separate pieces of information, separated by a comma: the string "Score:" and the integer 100.

When `print()` receives multiple pieces like this, it automatically displays them all on the same line, with a single space placed between them.

The output looks like this: `Score: 100` notice Python handled combining a string and a number for us here, without needing any type conversion.

Common Mistakes to Avoid:

- Forgetting one of the two quotation marks around a string Python will show a `SyntaxError` because it never finds where the text ends.
- Forgetting a closing parenthesis Python will wait for more input or show an error about an unexpected end of the line.
- Mixing up single quotes and double quotes so they don't match, like `print('Hi')` always start and end a string with the same type of quote mark.

□ **Mini Challenge: Say Hello Your Way**

Write three separate `print()` lines that introduce yourself: your name, your favorite animal, and your favorite color.

Run the program and check that all three lines appear in the correct order, top to bottom.

Try combining two pieces of information into a single `print()` call using a comma, the way we did with `Score: 100`.

Variables: Boxes in the Computer's Brain (and Comments)

So far, we've only asked Python to show us information once. But what if we want to store information so we can use it again and again? That's exactly what variables are for.

A variable is like a labeled box inside the computer's memory. You give the box a name, and you put something inside it. Later, whenever you use that name, Python opens the box and hands you whatever is inside.

Let's create our first variable:

```
player_name = "Ada"
print(player_name)
```

□ Here's What's Happening:

`player_name` is the name we chose for our box. We could have called it anything, but a name that describes what's inside like `player_name` makes our code much easier to understand later.

The equal sign (`=`) is called the assignment operator. In everyday math, an equal sign means "these two things are the same." In Python, it means something slightly different: "take whatever is on the right, and store it inside the box on the left."

"Ada" is the value we're storing a string containing the text Ada.

On the second line, `print(player_name)` tells Python to open the `player_name` box, take out whatever is inside, and display it. Python shows Ada, not the word `player_name`, because it looks inside the box rather than printing the label.

Variables can also change over time that's why they're called variables! Watch what happens here:

```
score = 0
print(score)
score = 10
print(score)
```

□ Here's What's Happening:

We create a variable called `score` and give it the value 0, because the player hasn't earned any points yet. As the game progresses, this number will increase.

`print(score)` opens the box and shows what's inside, which right now is 0.

`score = 10` replaces whatever was inside the `score` box with a new value, 10. The old value, 0, is gone forever a box can only hold one thing at a time.

The second `print(score)` now shows 10, because that's what is currently stored inside the box.

This idea changing a stored number as a game goes on is exactly how a score, a health bar, or a timer works in every video game you've ever played.

Assigning Multiple Variables at Once

Python offers a shortcut for creating several variables in a single line:

```
player_x, player_y, player_health = 0, 100, 50
print(player_x)
print(player_y)
print(player_health)
```

□ Here's What's Happening:

On the left side of the equal sign, we list three variable names separated by commas: `player_x`, `player_y`, `player_health`.

On the right side, we list three matching values, also separated by commas, in the same order: `0`, `100`, `50`.

Python matches them up in order: `player_x` becomes `0`, `player_y` becomes `100`, and `player_health` becomes `50` all created in a single, tidy line instead of three separate ones.

This shortcut is especially handy for game objects, which often need several related values, like an x position, a y position, and a health value, set up all at once.

Comments

Sometimes you want to leave yourself (or someone else reading your code) a little note that Python will completely ignore. That's what a comment is for.

```
# This variable keeps track of the player's score
score = 0
```

□ Here's What's Happening:

Any line that starts with a hashtag symbol (`#`) is a comment. Python skips right over it and does not try to run it as code.

Comments don't change what your program does they exist purely to help humans understand the code. Get in the habit of leaving short comments to explain tricky or important parts of your programs.

You can also place a comment at the end of a line of code, after the code itself:

```
health = 100 # starting health for a new player
```

□ Here's What's Happening:

Python runs `health = 100` exactly as normal, and then, upon reaching the `#` symbol, ignores everything after it on that same line.

This style is useful for adding a short clarifying note right next to the specific line it describes.

Rules for Naming Variables

You have a lot of freedom when naming your variables, but Python does have a few strict rules, plus some friendly style habits that make your code easier to read.

- Names can only contain letters, numbers, and underscores (`_`) no spaces or symbols like `@` or `!`.
- Names cannot start with a number (`score1` is fine, but `1score` is not allowed).
- Names are case-sensitive, meaning `Score` and `score` are treated as two completely different variables.
- Avoid Python's reserved words, like `print` or `if`, since those already mean something special to Python.
- By style convention, Python variable names use lowercase letters with underscores between words, like `player_health` instead of `PlayerHealth`.

□ Tip:

Choose descriptive names! `player_health` tells you exactly what's stored there, while a name like `x` or `data1` forces you to remember (or guess) what it means later.

Common Mistakes to Avoid:

- Using a variable before ever creating it Python will raise a `NameError` if you try to `print(score)` before ever writing `score = 0`.
- Accidentally creating two similarly-named variables, like `player_health` and `playerhealth`, and confusing yourself about which one you meant to update.

□ Mini Challenge: Build a Character Sheet

Create variables for `hero_name`, `hero_health`, and `hero_level`, storing whatever starting values you like.

Add a comment above each variable explaining what it represents.

Print all three variables, then change `hero_health` to a lower number (as if the hero got hurt) and print it again.

Try creating all three variables in a single line, using the multiple-assignment shortcut you just learned.

Data Types: What Goes in the Box?

Not everything you store in a variable is the same kind of thing. A player's name is different from a player's score, which is different from whether a door is locked or unlocked. Python calls these different kinds of information data types.

Here are the four data types you'll use constantly while building your game:

- String (str): text, written between quotation marks, like "Ada" or "Game Over".
- Integer (int): a whole number, with no decimal point, like 0, 10, or -5.
- Float (float): a number with a decimal point, like 3.14 or 98.6.
- Boolean (bool): a value that is only ever True or False perfect for yes/no questions like `is_game_over`.

```
player_name = "Ada"      # string
player_score = 100       # integer
player_speed = 4.5       # float
is_alive = True          # boolean
```

□ Here's What's Happening:

`player_name = "Ada"` stores text, so Python treats it as a string. The quotation marks are the signal that tells Python this is a string.

`player_score = 100` has no quotation marks and no decimal point, so Python treats it as an integer a whole number.

`player_speed = 4.5` includes a decimal point, so Python stores it as a float, which is useful whenever precision matters, like speed or distance.

`is_alive = True` is a boolean. Notice True is written with a capital T and no quotation marks that's how Python recognizes it as a boolean rather than a string.

You can check what data type a variable holds using Python's built-in `type()` function:

```
print(type(player_name))
print(type(player_score))
```

□ Here's What's Happening:

`type()` is another built-in function, just like `print()`. Instead of displaying its input, it looks at the input and tells you what kind of data type it is.

`print(type(player_name))` will show `<class 'str'>`, confirming that `player_name` holds a string.

`print(type(player_score))` will show `<class 'int'>`, confirming `player_score` holds an integer.

Why Data Types Matter

Data types matter because different types behave differently, especially with the + operator.

```
print(5 + 3)
print("5" + "3")
```

□ Here's What's Happening:

`print(5 + 3)` adds two integers together mathematically, producing 8.

`print("5" + "3")` looks similar, but because both "5" and "3" are strings (notice the quotation marks), the + operator glues them together as text instead of doing math, producing the string 53, not the number 8.

This is exactly why knowing your data types matters the very same + symbol behaves completely differently depending on whether it's working with numbers or strings.

□ Tip:

If you ever get an unexpected result from math in your code, the very first thing to check is whether your variables are actually the data type you think they are, using `type()`.

□ Mini Challenge: Type Detective

Create one variable of each data type: a string, an integer, a float, and a boolean.

Use `type()` and `print()` together to confirm each one is the data type you expect.

Predict what `print("2" + "2")` will display before running it, then check if you were right.

Interactivity: The `input()` Function

So far, our programs have only shown information to us they haven't asked us anything. Let's fix that. The `input()` function lets your program pause, ask the player a question, and wait for them to type an answer.

```
player_name = input("What is your name? ")
print("Welcome, " + player_name + "!")
```

□ Here's What's Happening:

`input("What is your name? ")` displays the message What is your name? on the screen and then pauses the program completely, waiting for the player to type something and press Enter.

Whatever the player types is handed back and stored inside the `player_name` variable just like we stored values before, except this time the value comes from the player instead of from us.

The plus sign (+) between strings glues them together into one longer string. This is called concatenation. So "Welcome, " + player_name + "!" joins the greeting, the player's typed name, and an exclamation point into a single message.

print(...) then displays that combined message. If the player typed Sam, the output would read: Welcome, Sam!

□ **Tip:**

Every value that comes from input() is stored as a string, even if the player types a number! We'll fix that in the very next section.

Type Conversion

Try this code and see what happens:

```
age = input("How old are you? ")
next_year = age + 1
print(next_year)
```

This will actually cause an error! Python will complain that it can't add a string and an integer together. That's because age holds a string (even if the player typed a number like "12"), and you cannot mix a string with an integer using +.

To fix it, we need to convert the string into an integer using the int() function:

```
age = input("How old are you? ")
age = int(age)
next_year = age + 1
print(next_year)
```

□ **Here's What's Happening:**

int(age) takes whatever is stored in age (a string, like "12") and converts it into a true integer, 12.

By writing age = int(age), we replace the old string value inside the age box with the new integer value.

Now next_year = age + 1 works perfectly, because we're adding two integers together, giving us 13.

print(next_year) displays the number 13.

You can convert data types the other way too, using str() to turn a number into a string, or float() to turn a whole number or string into a decimal number.

```
height_cm = 150
message = "Your height is " + str(height_cm) + " cm"
print(message)
```

□ Here's What's Happening:

`height_cm = 150` stores an integer.

`str(height_cm)` converts that integer into the string "150", which is necessary here because we're about to glue it together with other strings using `+`.

Without wrapping it in `str()`, Python would raise an error trying to add a string and an integer directly with `+`, just like we saw earlier with `age`.

□ Tip:

A very common error message you'll see while learning is: `TypeError: can only concatenate str (not "int") to str`. Whenever you see this, it means you tried to glue a string and a number together with `+` without converting one of them first.

Basic Arithmetic Operators

Python understands math, using symbols called arithmetic operators:

- `+` addition, like `health + 10`
- `-` subtraction, like `health - 5`
- `*` multiplication, like `score * 2`
- `/` division, like `total / players` (always gives a float)
- `//` integer (floor) division, like `total // players` (drops any remainder)
- `%` modulo, which gives you the remainder of a division, like `10 % 3`, which equals 1
- `**` exponent (power), like `2 ** 3`, which equals 8

```
health = 100
health = health - 25
print(health)
```

□ Here's What's Happening:

`health = 100` stores the starting health of our player.

`health = health - 25` looks inside the health box, takes out 100, subtracts 25 from it, and puts the new result, 75, back into the same box.

`print(health)` then shows 75 this exact pattern (`variable = variable - amount`) is how damage works in almost every game ever made.

□ Tip:

Python has a shortcut for this common pattern: `health -= 25` does the exact same thing as `health = health - 25`. The same shortcut works for `+=`, `*=`, and `/=` too.

The modulo operator (`%`) deserves special attention, since it's less familiar than the others but incredibly useful in games:

```
print(10 % 3)
print(7 % 2)
print(8 % 2)
```

□ **Here's What's Happening:**

10 % 3 asks, "if I divide 10 by 3, what's left over?" Since 3 goes into 10 three times (9), with 1 left over, the answer is 1.

7 % 2 divides 7 by 2, which goes in three times (6), leaving a remainder of 1 so 7 % 2 equals 1.

8 % 2 divides evenly, with nothing left over, so 8 % 2 equals 0.

This last pattern checking if `number % 2 == 0` is exactly how programmers check whether a number is even, which you'll actually use again in Week 2.

Simple Mini Projects

□ **Mini Challenge: Greeting Machine**

Ask the player for their name and their favorite color using `input()`.

Print a friendly, personalized message using both pieces of information.

□ **Mini Challenge: Tip Calculator**

Ask the player for a bill total using `input()`, and convert it to a float.

Calculate a 15% tip by multiplying the bill total by 0.15.

Print the tip amount and the total amount including the tip.

Debugging Basics

A bug is a mistake in your code that causes it to behave incorrectly or crash.

Debugging is the process of finding and fixing bugs. Every programmer, no matter how experienced, spends time debugging it's a normal and important part of coding.

When Python hits an error, it prints something called a traceback. It looks scary at first, but it's actually incredibly helpful. Here's how to read one:

- Read the very last line first it usually tells you the type of error and a short description.
- Look at the line number mentioned in the traceback that's exactly where Python got confused.
- Common beginner errors include: forgetting a closing quotation mark or parenthesis, misspelling a variable name, or mixing data types (like a string and an integer) without converting them first.

Let's look at three common error types you're likely to meet early on:

- **SyntaxError** Python couldn't even understand the structure of your code, often from a missing colon, quote, or parenthesis.
- **NameError** you tried to use a variable that was never created, often because of a typo in the variable's name.
- **TypeError** you tried to combine two data types that don't work together directly, like a string and an integer with +.

□ **Tip:**

When you get an error message, don't panic! Read it slowly, find the line number, and check that line carefully. Most bugs are simple typos once you find them.

Supercharging Print with f-strings

Gluing strings together with + signs works, but it gets messy quickly, especially when you're mixing text with numbers. Python has a much cleaner tool called an f-string.

```
player_name = "Ada"
player_score = 100
print(f"{player_name} has {player_score} points!")
```

□ **Here's What's Happening:**

Putting the letter f directly before the opening quotation mark tells Python, "this is a special f-string look inside for curly braces."

Anything inside curly braces { } is treated as a variable (or even a small calculation) rather than plain text. Python looks up the value and inserts it directly into the string. {player_name} is replaced with Ada, and {player_score} is replaced with 100, so the final printed message reads: Ada has 100 points!

Notice we didn't need any + signs or type conversions f-strings handle mixing text and numbers automatically.

You can even do small calculations directly inside the curly braces:

```
health = 40
max_health = 100
print(f"Health: {health}/{max_health} ({health / max_health * 100}%)")
```

□ **Here's What's Happening:**

Inside the third pair of curly braces, health / max_health * 100 is calculated first, giving us the player's health as a percentage, and then that calculated result is inserted directly into the string.

This shows that f-strings aren't limited to just displaying variables as-is they can display the result of an entire expression.

❑ Tip:

From this point on in the guide, we'll use f-strings whenever we combine text and variables together they're the modern, clean way to build messages in Python.

Math and Type Casting

Let's bring everything together with one more important idea: sometimes you need to deliberately convert, or cast, a value from one data type to another to make your math work correctly. This is called type casting.

```
apples = "5"  
apples = int(apples)  
total_fruit = apples + 3  
print(total_fruit)
```

❑ Here's What's Happening:

`apples = "5"` stores the text "5" notice the quotation marks. Even though it looks like a number, Python currently treats it as a string.

`int(apples)` converts that string into the true integer 5, and we store the result back into `apples`.

`total_fruit = apples + 3` now adds two integers together, giving us 8.

`print(total_fruit)` displays 8. If we had skipped the `int()` conversion, this line would have caused an error.

Here's a quick reference for the type casting functions you'll use most:

- `int(value)` converts to a whole number (drops anything after a decimal point).
- `float(value)` converts to a decimal number.
- `str(value)` converts to text, useful when combining numbers into a message with `+` instead of an f-string.

Be careful: converting a float to an int doesn't round it simply chops off everything after the decimal point.

```
print(int(9.9))  
print(int(9.1))
```

❑ Here's What's Happening:

`int(9.9)` does not round up to 10. It truncates (cuts off) the decimal portion entirely, giving us 9.

`int(9.1)` works the same way, also giving us 9, even though 9.1 is much closer to 9 than to 10.

If you actually want mathematical rounding, Python has a separate built-in function for that: `round(9.9)` would correctly give you 10.

□ Mini Challenge: Average Score Calculator

Ask the player for three test scores using `input()`, converting each one to a float.

Add the three scores together and divide by 3 to find the average.

Print the result using an f-string, like `f"Your average score is {average}"`.

Week 1 Recap

Let's review everything you've learned this week in one place.

- `print()` displays information on the screen; `input()` pauses the program and asks the player for information.
- Variables are labeled storage boxes, created with the assignment operator (`=`).
- The four core data types are string (`str`), integer (`int`), float (`float`), and boolean (`bool`).
- Comments (`#`) are notes for humans that Python ignores completely.
- Arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) let Python do math.
- Type casting (`int()`, `float()`, `str()`) converts a value from one data type to another.
- f-strings, written as `f"...{variable}..."`, are the cleanest way to combine text and variables.
- Reading error messages carefully especially the last line and the line number is the first step of debugging.

Check Your Understanding

20. What is the difference between `=` and `==`? (Hint: you'll use `==` starting next week!)
21. Why does `input()` always return a string, even if the player types a number?
22. What will `print(type(4.0))` display, and why?
23. What is wrong with this line of code: `print("Score: " + 10)`? How would you fix it two different ways?
24. What does the `%` operator calculate, and how could it be used to check whether a number is even?

You've officially finished Week 1! You can now store information in variables, understand the four basic data types, ask the player questions, do math, and read your first error messages. Every single one of these skills will show up again and again as we build our game.

WEEK 2

Control Flow & Loops

Teaching Python to think and repeat itself

Comparison Operators

Last week, your programs always ran the exact same way, top to bottom, every single time. This week, we teach Python how to make decisions to do one thing if something is true, and a different thing if it's not. Before we can make decisions, though, Python needs a way to compare things.

Comparison operators ask a yes-or-no question and produce a boolean answer: True or False.

- `==` is equal to
- `!=` is not equal to
- `>` greater than
- `<` less than
- `>=` greater than or equal to
- `<=` less than or equal to

```
health = 50
print(health > 0)
print(health == 100)
```

□ Here's What's Happening:

`health = 50` stores our starting value.

`health > 0` asks Python a question: "is the value inside health greater than 0?" Since 50 is greater than 0, this expression evaluates to True.

`print(health > 0)` displays True.

`health == 100` asks a different question: "is health exactly equal to 100?" Since health holds 50, not 100, this evaluates to False.

□ Tip:

Don't mix up `=` and `==`! A single equal sign (`=`) stores a value. A double equal sign (`==`) compares two values. This is one of the most common beginner mix-ups, so watch for it.

Comparison operators work on strings too, not just numbers:

```
answer = "yes"
print(answer == "yes")
print(answer != "no")
```

□ Here's What's Happening:

`answer == "yes"` compares the text stored in `answer` with the text `"yes"`. Since they match exactly, this is `True`.

`answer != "no"` checks whether `answer` is not equal to `"no"`. Since `answer` holds `"yes"`, not `"no"`, this is also `True`.

Note that string comparisons are case-sensitive: `"Yes"` and `"yes"` would NOT be considered equal, since capital and lowercase letters are treated as different characters.

Real-World Connection: every single time a game checks "has the player reached the exit?" or "is the password correct?", it's using a comparison operator just like these, feeding the result straight into a decision.

□ Mini Challenge: Password Checker

Store a secret password in a variable, like `secret = "dragon123"`.

Ask the player to type a guess using `input()`.

Use `==` to compare the guess to the secret, and `print(...)` whether they matched.

Boolean Logic: The Brains of the Game

Every decision your game will ever make has the player run out of lives? did the ship hit an asteroid? is it game over? boils down to a boolean value: `True` or `False`. That's why booleans are often called the brains of the game.

Think about a light switch. It only has two states: on or off. There's no in-between.

Booleans work exactly the same way a value is either `True` or `False`, never both, and never something else.

```
is_game_over = False
has_key = True
print(is_game_over)
print(has_key)
```

□ Here's What's Happening:

`is_game_over = False` creates a boolean variable that starts as `False`, since the game has just begun.

`has_key = True` creates another boolean, showing the player currently has a key item.

Notice that True and False are capitalized and have no quotation marks that's how Python tells them apart from strings like "True".

`print(is_game_over)` and `print(has_key)` simply display the current value stored in each box.

As your game runs, these boolean values will flip back and forth `is_game_over` will switch to True the moment the player loses, and `has_key` might switch to True the moment they pick up an item. Booleans are how your game keeps track of its own state.

You can also store the result of a comparison directly inside a boolean variable, instead of typing True or False by hand:

```
health = 0
is_defeated = health <= 0
print(is_defeated)
```

□ Here's What's Happening:

`health <= 0` is evaluated first, producing either True or False based on the current value of `health`.

Since `health` is 0, the comparison `health <= 0` evaluates to True, and that result the boolean value itself, not the comparison text is what gets stored inside `is_defeated`.

`print(is_defeated)` then displays True.

□ Tip:

Naming boolean variables starting with `is_` or `has_`, like `is_game_over` or `has_key`, is a common style habit that makes it instantly obvious the variable holds True or False.

You can also flip a boolean directly, which is a handy pattern for anything that toggles on and off, like a paused state:

```
is_paused = False
is_paused = not is_paused
print(is_paused)

is_paused = not is_paused
print(is_paused)
```

□ Here's What's Happening:

`not is_paused` flips whatever `is_paused` currently holds since it starts as False, not False becomes True.

Storing that flipped result back into `is_paused` means each time this line runs, the value toggles to the opposite of what it just was.

The first print shows True, and after flipping again, the second print shows False exactly the on/off behavior a pause button needs.

The IF Statement

Now that we can create True/False questions, let's use them to control what our program actually does. The if statement tells Python: "only run this block of code if this condition is True."

```
health = 0

if health <= 0:
    print("Game Over!")
```

□ Here's What's Happening:

if health <= 0: starts the decision. Python checks whether health is less than or equal to 0. Since health is currently 0, this condition is True.

The colon (:) at the end of the if line tells Python, "an indented block of code is coming next only run it if the condition above is True."

print("Game Over!") is indented underneath the if line. Indentation (the spaces before the code) is how Python knows this line belongs inside the if statement, not outside of it. Python is very strict about this get in the habit of using 4 spaces (or one Tab) for every level of indentation.

Because health <= 0 was True, Python runs the indented print line and displays Game Over!. If health had been greater than 0, Python would have skipped that line entirely.

□ Tip:

Forgetting the colon (:) or the indentation are two of the most common beginner mistakes with if statements. If Python gives you an IndentationError, check your spacing carefully.

Elif and Else

What if we want to check several possibilities, one after another? That's what elif (short for "else if") is for. And else catches anything that didn't match any of the earlier conditions.

```
health = 45

if health > 75:
    print("You are in great shape!")
elif health > 25:
```

```
    print("You are hurt, be careful.")
else:
    print("Critical health! Find a potion!")
```

□ Here's What's Happening:

Python checks the conditions from top to bottom, in order, and stops at the very first one that is True.

if health > 75: checks first. Since health is 45, this is False, so Python skips this block entirely and moves to the next check.

elif health > 25: is checked next. Since 45 is greater than 25, this condition is True, so Python runs the indented line under it, printing You are hurt, be careful. and then it stops checking any further conditions.

The final else: block would only run if none of the conditions above it were True. Since our elif already matched, this else block is skipped entirely.

□ Tip:

You can chain as many elif statements as you need between an if and an else, but there can only ever be one if to start and, at most, one else at the very end.

Common Mistakes to Avoid:

- Writing several separate if statements when you meant elif this causes Python to check every condition independently instead of stopping at the first match, which can lead to multiple messages printing when you only wanted one.
- Forgetting that order matters: if you check health > 25 before health > 75, a health value of 90 would incorrectly match the first, more general condition instead of the more specific one.

Nested Conditions

Sometimes a decision depends on another decision. You can place an if statement inside another if statement this is called nesting.

```
has_key = True
health = 60

if health > 0:
    if has_key:
        print("You unlock the door and escape!")
    else:
        print("You need a key to open this door.")
```

□ Here's What's Happening:

The outer `if health > 0`: first checks whether the player is even still alive. Since `health` is 60, this is `True`, so Python enters this block.

Inside that block, a second `if has_key`: checks whether the player is carrying a key. Notice this inner `if` is indented one extra level, showing it lives inside the outer `if`. Since `has_key` is `True`, Python runs the innermost `print` line, displaying the escape message. If `has_key` had been `False`, the `else` directly beneath it would have run instead.

If `health` had not been greater than 0 in the first place, Python would have skipped the entire outer block, and neither inner message would ever print.

□ **Tip:**

Nested conditions can get confusing quickly if you go more than two or three levels deep. When that happens, using `and` (which we'll cover next) is often a cleaner alternative.

Logical Operators (`and`, `or`, `not`)

Logical operators let you combine multiple `True/False` conditions into a single, more powerful condition.

- `and` the whole condition is `True` only if both sides are `True`.
- `or` the whole condition is `True` if at least one side is `True`.
- `not` flips a condition, turning `True` into `False` and `False` into `True`.

```
health = 80
has_shield = True

if health > 50 and has_shield:
    print("You feel safe and strong.")
```

□ **Here's What's Happening:**

`health > 50` checks to `True`, since 80 is greater than 50.

`has_shield` is already `True`.

The `and` operator requires both sides to be `True` for the entire condition to be `True`. Since both are `True`, Python runs the indented `print` line.

If either `health` had dropped to 50 or below, or `has_shield` had been `False`, this `and` condition would become `False`, and the message would not print.

Here's how `or` and `not` behave differently:

```
has_sword = False
has_bow = True
```

```
if has_sword or has_bow:
    print("You are armed!")

if not has_sword:
    print("You have no sword equipped.")
```

□ Here's What's Happening:

has_sword or has_bow only needs one side to be True for the whole condition to be True. Even though has_sword is False, has_bow is True, so the overall condition is True, and "You are armed!" prints.

not has_sword flips the value of has_sword. Since has_sword is False, not has_sword becomes True, so the message about having no sword prints.

You can also combine and and or together in a single condition, using parentheses to make the order clear, just like in math class:

```
health = 30
has_shield = False
has_potion = True

if health > 50 or (has_shield and has_potion):
    print("You can handle this fight.")
else:
    print("Retreat and heal up first!")
```

□ Here's What's Happening:

The parentheses around (has_shield and has_potion) tell Python to evaluate that pair together first, as its own single True/False result.

Since has_shield is False, has_shield and has_potion evaluates to False, no matter what has_potion is.

Then health > 50 or False is checked. Since health is only 30, health > 50 is also False, making the entire condition False.

Because the whole condition is False, Python runs the else block instead, printing the retreat message.

□ Mini Challenge: Traffic Light Decisions

Create a variable called light_color storing the string "red", "yellow", or "green".

Write an if / elif / else chain that prints "Stop!", "Slow down!", or "Go!" depending on the color.

Add a second variable is_pedestrian_crossing, and use and to print a special "Wait for pedestrians!" message when the light is green AND someone is crossing.

Loops: Don't Repeat Yourself

Imagine you wanted to print the numbers 1 through 100. Would you really want to type `print()` one hundred separate times? Of course not! Loops let us repeat a block of code automatically, without retyping it over and over. This idea of never repeating code you don't have to is such an important habit in programming that it has its own nickname: DRY, short for "Don't Repeat Yourself."

While Loops

A while loop repeats its indented block of code for as long as a condition remains True.

```
countdown = 5

while countdown > 0:
    print(countdown)
    countdown = countdown - 1

print("Blast off!")
```

□ Here's What's Happening:

`countdown = 5` sets our starting value.

`while countdown > 0:` checks the condition. Since `countdown` is 5, which is greater than 0, this is True, so Python enters the loop's indented block.

`print(countdown)` displays the current value, 5.

`countdown = countdown - 1` reduces `countdown` by 1, making it 4.

Python then jumps back up to the `while` line and checks the condition again. Since `countdown` is still greater than 0, it repeats the whole indented block. This continues until `countdown` finally becomes 0.

The moment `countdown > 0` becomes False, Python exits the loop completely and moves on to the un-indented line below it, printing Blast off!.

□ Tip:

Be very careful with while loops! If you forget to change the condition inside the loop (like forgetting `countdown = countdown - 1`), the condition will never become False, and your program will get stuck forever in what's called an infinite loop. If this happens, use the Stop button (red square) in PyCharm to force the program to quit.

A while loop is also perfect for repeating something until the player gives valid input:

```
answer = ""
```

```
while answer != "quit":  
    answer = input("Type a command (or 'quit' to stop): ")  
    print(f"You typed: {answer}")
```

□ Here's What's Happening:

`answer = ""` creates an empty string to start, just so the variable exists before the loop begins checking it.

`while answer != "quit":` keeps the loop running for as long as the player has not typed the exact word quit.

Inside the loop, `answer = input(...)` asks the player for a new command every single time, replacing the old value stored in `answer`.

The moment the player types quit, the condition `answer != "quit"` becomes False, and the loop stops on its own.

For Loops, Range, Break, and Continue

A for loop repeats its block a specific number of times, often working through a sequence of values. The `range()` function is a common way to generate a sequence of numbers to loop through.

```
for number in range(5):  
    print(number)
```

□ Here's What's Happening:

`range(5)` generates a sequence of numbers starting at 0 and stopping right before 5, giving us 0, 1, 2, 3, 4 five numbers in total.

`for number in range(5):` tells Python, "one at a time, take each value out of that sequence, store it in the variable `number`, and run the indented block below."

`print(number)` runs five separate times once for each value in the sequence displaying 0, then 1, then 2, then 3, then 4, each on its own line.

Once Python has gone through every value in the sequence, the loop ends automatically there's no need to manually change a condition, unlike a while loop.

`range()` can also take a starting point and a step size:

```
for number in range(2, 11, 2):  
    print(number)
```

□ Here's What's Happening:

`range(2, 11, 2)` means "start at 2, stop before 11, and count up by 2 each time."

This produces the sequence 2, 4, 6, 8, 10 every even number from 2 through 10.

This three-number version of `range()` `range(start, stop, step)` is extremely useful whenever you need to count by something other than 1.

Two special commands let you control a loop's behavior from the inside: `break` stops the loop completely, and `continue` skips the rest of the current pass and jumps to the next one.

```
for number in range(10):  
    if number == 5:  
        break  
    print(number)
```

□ Here's What's Happening:

This loop would normally count from 0 to 9, but we've added a condition inside it.

`if number == 5:` checks, on every pass, whether we've reached the number 5.

`break` immediately stops the entire loop the instant number equals 5. Python doesn't finish that pass, and it doesn't go back to check the loop condition again. It just exits.

Because of this, the output only ever shows 0, 1, 2, 3, 4 the numbers 5 through 9 are never reached.

```
for number in range(6):  
    if number % 2 == 0:  
        continue  
    print(number)
```

□ Here's What's Happening:

`number % 2 == 0` checks whether number is evenly divisible by 2, in other words, whether it's an even number.

`continue` immediately skips the rest of the current pass through the loop and jumps straight to the next value the print line below it never runs for that particular pass.

Because every even number triggers `continue`, only the odd numbers ever reach the print line, so the output is 1, 3, 5.

Common Mistakes to Avoid:

- Confusing while loops (which repeat based on a condition) with for loops (which repeat a set number of times, or once per item in a sequence).
- Forgetting that `range(5)` stops before 5, producing 0 through 4, not 1 through 5.
- Placing `break` or `continue` outside of any loop, which causes a `SyntaxError`, since Python only understands those commands inside a loop's indented block.

Nested Loops

Just like if statements, loops can be nested inside other loops. A nested loop is a loop that runs completely, start to finish, on every single pass of an outer loop perfect for anything grid-shaped.

```
for row in range(3):
    for column in range(4):
        print("*", end="")
    print()
```

□ **Here's What's Happening:**

The outer loop, `for row in range(3):`, runs its indented block exactly 3 times, once for each row we want to draw.

On each of those 3 passes, the entire inner loop, `for column in range(4):`, runs completely from start to finish, printing four asterisks in a row.

`print("*", end="")` displays an asterisk without moving to a new line afterward the `end=""` argument overrides `print()`'s normal habit of adding a line break, letting our asterisks sit side by side.

The plain `print()` after the inner loop (with nothing inside its parentheses) adds a single line break once each row of asterisks is finished, moving us down to start the next row.

The result is a neat 3-row, 4-column grid of asterisks nested loops are exactly how you'd build a grid-based game board, like tic-tac-toe or a dungeon map.

□ **Tip:**

Whenever you catch yourself thinking "I need to repeat this whole repeating thing, several times," that's a strong signal you need a nested loop.

□ **Mini Challenge: Countdown and Cheer**

Write a while loop that counts down from 10 to 1, printing each number.

Write a for loop using `range()` that prints every multiple of 3 between 0 and 30.

Add a break statement that stops the multiples-of-3 loop the moment it reaches 15.

Use a nested loop to print a 5x5 grid of asterisks.

Adding Chaos: The Random Module

Real games rarely behave exactly the same way every time you play them enemies wander unpredictably, loot drops are surprises, and dice rolls keep things interesting. To add that unpredictability to our own programs, we use Python's random module.

A module is a bundle of extra tools that isn't loaded into Python automatically you have to ask for it first, using the `import` keyword.

```
import random

dice_roll = random.randint(1, 6)
print(dice_roll)
```

□ Here's What's Happening:

`import random` loads the entire random module, giving us access to all of its tools. This line always goes at the very top of your file.

`random.randint(1, 6)` calls the `randint` function that lives inside the random module (that's why we write `random` dot `randint`). It picks a random whole number between 1 and 6, including both 1 and 6 themselves.

`dice_roll = ...` stores whatever random number was chosen inside our `dice_roll` variable.

`print(dice_roll)` displays that number. If you run this program multiple times, you'll notice the number is different (or at least could be different) each time that's randomness in action!

Here are a few other extremely useful tools from the random module:

- `random.randint(a, b)` a random whole number between a and b, inclusive.
- `random.random()` a random decimal number between 0.0 and 1.0.
- `random.uniform(a, b)` a random decimal number between a and b, useful for things like a random speed or a random angle.
- `random.choice(a_list)` randomly picks one item out of a list (we'll learn about lists in Week 3!).
- `random.shuffle(a_list)` rearranges every item in a list into a random new order.

Combining random numbers with an if statement is an incredibly common game pattern:

```
import random

roll = random.randint(1, 100)

if roll <= 10:
    print("Critical hit!")
else:
    print("Normal attack.")
```

□ Here's What's Happening:

`random.randint(1, 100)` simulates a percentage-based chance by picking a random number between 1 and 100.

if roll <= 10: checks whether that random roll landed in the bottom 10 possible values, meaning we've built a 10% chance of a critical hit.

This exact pattern rolling a random number and checking whether it falls under a threshold is how loot drop rates, critical hit chances, and random events work in countless real video games.

□ **Tip:**

In Week 4, random will become one of your best friends it's how enemies will move unpredictably and how surprises will appear in your game.

Putting It All Together: A Number Guessing Game

Let's combine everything from this entire week comparisons, if/elif/else, a while loop, and randomness into one complete mini-game.

```
import random

secret_number = random.randint(1, 20)
guess = 0
attempts = 0

while guess != secret_number:
    guess = int(input("Guess a number between 1 and 20: "))
    attempts += 1

    if guess < secret_number:
        print("Too low!")
    elif guess > secret_number:
        print("Too high!")
    else:
        print(f"You got it in {attempts} tries!")
```

□ **Here's What's Happening:**

secret_number = random.randint(1, 20) picks a hidden number between 1 and 20 for the player to guess, using the random module.

guess = 0 and attempts = 0 set up two starting variables: a placeholder guess that definitely won't match the secret number yet, and a counter to track how many tries it takes.

while guess != secret_number: keeps the loop running for as long as the player hasn't guessed correctly.

guess = int(input(...)) asks the player for a number, converting their typed string into a real integer, exactly like we practiced in Week 1.

attempts += 1 increases our attempt counter by one, every single time through the loop.

The if / elif / else chain compares the guess to the secret number and gives the player a helpful hint, using the comparison operators from earlier this week.

The moment guess equals secret_number, the else branch congratulates the player, and since the while loop's condition (guess != secret_number) is now False, the loop ends right after this pass.

□ **Tip:**

This small game uses ideas from every single section in Week 2. If any part of it feels unclear, that's a great sign of exactly which section to revisit before moving on.

□ **Mini Challenge: Magic 8-Ball**

Import the random module.

Ask the player to type a yes/no question using input().

Use random.randint(1, 3) to pick a number, then use if / elif / else to print "Yes", "No", or "Ask again later" depending on the number.

□ **Mini Challenge: Improve the Guessing Game**

Add a maximum of 6 attempts if the player runs out of tries without guessing correctly, print the secret number and a "Better luck next time" message.

Add a running list of every guess the player has made so far (this is a sneak peek at lists, coming in Week 3!).

Putting It All Together, Again: Rock, Paper, Scissors

Let's practice everything once more with a second classic game, this time one that repeats for several rounds and keeps a running score a great warm-up for the scoring systems we'll build in Week 4.

```
import random

choices = ["rock", "paper", "scissors"]
player_wins = 0
computer_wins = 0

for round_number in range(3):
    player_choice = input("Choose rock, paper, or scissors: ")
    computer_choice = random.choice(choices)
    print(f"Computer chose {computer_choice}")

    if player_choice == computer_choice:
        print("It's a tie!")
```

```

    elif (player_choice == "rock" and computer_choice ==
"scissors") or \
        (player_choice == "paper" and computer_choice ==
"rock") or \
        (player_choice == "scissors" and computer_choice ==
"paper"):
        print("You win this round!")
        player_wins += 1
    else:
        print("Computer wins this round!")
        computer_wins += 1

print(f"Final score  You: {player_wins}, Computer:
{computer_wins}")

```

□ Here's What's Happening:

`choices = ["rock", "paper", "scissors"]` previews a list, which you'll learn all about next week for now, just know it's a way to group several related strings together so `random.choice()` can pick one.

`player_wins = 0` and `computer_wins = 0` set up two counters, similar to the score variable from Week 1, both starting at zero before any rounds are played.

`for round_number in range(3):` repeats everything inside it exactly 3 times, once per round, using the for loop pattern from earlier this week.

`random.choice(choices)` picks one random item from our list of three options the computer's move for this round.

The `elif` condition checks all three ways the player could win, joined together with `or`. The backslash (`\`) at the end of a line lets us continue writing a single long line of code onto the next line, purely for readability Python treats it as if it were all on one line.

Depending on which branch of the `if/elif/else` runs, we increase either `player_wins` or `computer_wins` by one, using the `+=` shortcut from Week 1.

After all 3 rounds finish, the loop ends automatically, and the final f-string reports the overall results.

□ Tip:

This game combines a for loop, `input()`, `random.choice()`, a chained `elif` with combined `and/or` logic, and running counters nearly everything from this entire week in one place. If any single piece feels fuzzy, this is a great program to experiment with.

□ Mini Challenge: Improve Rock, Paper, Scissors

Change the number of rounds from 3 to 5.

Add a check using `.lower()` on `player_choice` (a string method, similar to the ones we've used) so that typing "ROCK" or "Rock" still works correctly.

After the loop, add an `if/elif/else` that declares an overall winner by comparing `player_wins` to `computer_wins`.

Week 2 Recap

- Comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) produce a boolean answer: True or False.
- Booleans track the state of your game things that are either true or false, with nothing in between.
- `if`, `elif`, and `else` control which block of code actually runs, based on a condition.
- Nested conditions place an `if` inside another `if`, for decisions that depend on other decisions.
- Logical operators (`and`, `or`, `not`) combine or flip conditions, and can be grouped with parentheses.
- `while` loops repeat based on a condition remaining True; `for` loops repeat a set number of times, often using `range()`.
- `break` exits a loop immediately; `continue` skips to the next pass through the loop.
- Nested loops repeat an entire inner loop once for every pass of an outer loop great for grids.
- The `random` module adds unpredictability, using tools like `random.randint()`, `random.uniform()`, and `random.choice()`.

Check Your Understanding

25. What's the difference between `elif` and writing a second, separate `if` statement?
26. Write a condition using `and` that checks whether a player has more than 0 health AND is not carrying a cursed item.
27. What values does `range(3, 10, 3)` produce, and why?
28. What is an infinite loop, and what usually causes one by accident?
29. How would you use `random.randint()` to simulate a 25% chance of something happening?
30. In the number guessing game, why is `int()` needed around `input()` before comparing the guess to the secret number?

That wraps up Week 2! Python can now make decisions with `if` statements, repeat actions with loops, and surprise you with randomness. These three tools decisions, repetition, and randomness are the beating heart of every video game.

WEEK 3

Functions, Data & Graphics

Organizing your code and drawing your first pictures

Functions: Creating Your Own Commands

You've already used built-in functions like `print()`, `input()`, `int()`, and `type()`. This week, you'll learn to build your own functions — custom commands that do exactly what you want, whenever you want, without retyping the same code again and again.

```
def greet_player():  
    print("Welcome, brave adventurer!")  
    print("Your quest begins now.")  
  
greet_player()
```

□ Here's What's Happening:

`def` is a keyword that stands for "define." It tells Python, "I'm about to create a brand-new function."

`greet_player` is the name we chose for our function — good function names describe an action, which is why they often sound like verbs.

The parentheses `()` right after the name are required, even when empty — they're where a function's inputs would go, if it had any.

The colon `:` and the indented lines beneath it make up the function's body — the block of code that runs every time this function is called.

Simply defining a function doesn't run it! The last line, `greet_player()`, is what actually calls the function, telling Python, "run everything inside `greet_player` now." Only then do the two `print` lines execute.

□ Tip:

You can call the same function as many times as you like. Try adding `greet_player()` a second time at the bottom of the file and see the greeting print twice.

Common Mistakes to Avoid:

- Defining a function but forgetting to ever call it — Python will not complain, but nothing inside the function will ever run.

- Forgetting the parentheses when calling a function, like writing `greet_player` instead of `greet_player()` this refers to the function itself rather than running it.

Parameters and Arguments

Functions become far more useful when they can accept information from whoever calls them. A parameter is a placeholder name listed inside a function's parentheses, and an argument is the actual value you supply when calling the function.

```
def greet_player(name):  
    print(f"Welcome, {name}!")  
  
greet_player("Ada")  
greet_player("Sam")
```

□ Here's What's Happening:

`name`, written inside the parentheses in the `def` line, is a parameter a placeholder that will be filled in with whatever value is passed in later.

Inside the function body, `f"Welcome, {name}!"` uses that parameter just like any other variable.

`greet_player("Ada")` calls the function and supplies "Ada" as the argument. Python temporarily stores "Ada" inside the `name` parameter for this one call, and the function prints `Welcome, Ada!`.

`greet_player("Sam")` calls the exact same function again, but this time with a different argument, so it prints `Welcome, Sam!` instead. The function's code never changed only the input did.

You can give a function as many parameters as it needs, separated by commas:

```
def introduce(name, favorite_game):  
    print(f"{name} loves playing {favorite_game}!")  
  
introduce("Ada", "space adventures")
```

□ Here's What's Happening:

This function has two parameters, `name` and `favorite_game`, listed in order and separated by a comma.

When we call `introduce("Ada", "space adventures")`, Python matches the arguments to the parameters in the same order: `name` becomes "Ada", and `favorite_game` becomes "space adventures".

The function then prints `Ada loves playing space adventures!` using both pieces of information at once.

You can also give a parameter a default value, which is used automatically if the caller doesn't provide one:

```
def greet_player(name="adventurer"):
    print(f"Welcome, {name}!")

greet_player()
greet_player("Ada")
```

□ Here's What's Happening:

`name="adventurer"` gives the `name` parameter a default value of "adventurer", which Python will use automatically if no argument is supplied.

`greet_player()` is called with no arguments at all, so Python falls back on the default, printing `Welcome, adventurer!`.

`greet_player("Ada")` supplies its own argument, which overrides the default, printing `Welcome, Ada!` instead.

Returning Data

Sometimes a function shouldn't print anything itself instead, it should calculate something and hand the result back to whoever called it. That's what the `return` keyword is for.

```
def add_numbers(a, b):
    result = a + b
    return result

total = add_numbers(4, 7)
print(total)
```

□ Here's What's Happening:

`add_numbers(a, b)` is defined with two parameters, `a` and `b`.

Inside the function, `result = a + b` calculates the sum and stores it in a local variable called `result`, which only exists while the function is running.

`return result` is the key line: it hands the value stored in `result` back out of the function, ending the function immediately.

`total = add_numbers(4, 7)` calls the function with `a = 4` and `b = 7`. The function calculates 11 and returns it, and that returned value 11 gets stored in our `total` variable, exactly as if we had written `total = 11`.

`print(total)` then displays 11.

□ Tip:

A function can print things without returning anything, return something without printing anything, or do both but keep them straight in your head, because a function's return value is the only thing you can store in a variable afterward.

A function can also return early, based on a condition this is a very common pattern:

```
def check_level_up(experience):  
    if experience >= 100:  
        return True  
    return False  
  
print(check_level_up(120))  
print(check_level_up(40))
```

□ **Here's What's Happening:**

if experience >= 100: checks whether the player has enough experience to level up.

If that condition is True, return True immediately ends the function and hands back True the second return False line never even runs in that case.

If the condition was False, Python skips the if block entirely and reaches return False at the bottom, handing back False instead.

check_level_up(120) returns True, since 120 is at least 100; check_level_up(40) returns False, since 40 is less than 100.

One more useful habit: a function can call other functions, and use their returned values immediately.

```
def double(number):  
    return number * 2  
  
def add_ten(number):  
    return number + 10  
  
result = add_ten(double(5))  
print(result)
```

□ **Here's What's Happening:**

Python evaluates double(5) first, since it's the innermost function call, giving us 10.

That returned value, 10, is immediately handed to add_ten(...) as its argument, giving us 20.

result = ... stores this final value, 20, and print(result) displays it.

Combining small, focused functions like this each one doing a single clear job is a powerful habit that makes larger programs much easier to understand and fix.

□ **Mini Challenge: Health Potion Function**

Write a function called `heal(current_health, potion_strength)` that adds `potion_strength` to `current_health` and returns the new total.

Call the function with different starting health values and potion strengths, storing and printing each result.

Add a default value of 20 for `potion_strength`, and test calling the function with just one argument.

Data Structures: Lists and Dictionaries

A single variable can only hold one value at a time. But what if you need to keep track of an entire team of enemies, a whole inventory of items, or a full leaderboard of players? For that, we need data structures that can hold many values at once.

Lists

A list is an ordered collection of values, stored in a single variable, written between square brackets.

```
inventory = ["sword", "shield", "potion"]
print(inventory)
print(inventory[0])
```

□ Here's What's Happening:

`inventory = ["sword", "shield", "potion"]` creates a list containing three strings, separated by commas, all wrapped in square brackets.

`print(inventory)` displays the entire list at once, exactly as it's stored: `['sword', 'shield', 'potion']`.

`inventory[0]` uses square brackets with a number inside to grab one specific item from the list. This number is called an index.

Very important: Python lists start counting from 0, not 1! So `inventory[0]` refers to the first item, "sword", not the second one.

You can also count backward from the end of a list using negative indexes:

```
inventory = ["sword", "shield", "potion"]
print(inventory[-1])
print(inventory[-2])
```

□ Here's What's Happening:

`inventory[-1]` refers to the very last item in the list, no matter how long the list is in this case, "potion".

`inventory[-2]` refers to the second-to-last item, "shield".

Negative indexing is especially handy when you don't know (or don't want to calculate) exactly how many items are in a list.

Lists are changeable (we say mutable) you can add, remove, and update items after creating them.

```
inventory = ["sword", "shield"]
inventory.append("potion")
print(inventory)
```

```
inventory.remove("shield")
print(inventory)
```

□ Here's What's Happening:

`inventory.append("potion")` adds "potion" to the very end of the list. The dot between `inventory` and `append` tells Python, "use the append tool that belongs to this specific list."

After the append, printing `inventory` shows all three items: `['sword', 'shield', 'potion']`.

`inventory.remove("shield")` searches the list for "shield" and deletes it.

The final print shows the list with "shield" gone: `['sword', 'potion']`.

You can also check how many items are in a list using the `len()` function, and check whether an item exists using the `in` keyword:

```
inventory = ["sword", "shield", "potion"]
print(len(inventory))
print("shield" in inventory)
print("bow" in inventory)
```

□ Here's What's Happening:

`len(inventory)` counts how many items are currently in the list, giving us 3.

`"shield" in inventory` checks whether the string "shield" appears anywhere in the list, returning `True` since it does.

`"bow" in inventory` checks the same way, returning `False` since there is no "bow" in the list.

You'll also use loops constantly with lists, to check or display every item one at a time:

```
inventory = ["sword", "shield", "potion"]

for item in inventory:
    print(f"You have a {item}.")
```

□ Here's What's Happening:

for item in inventory: tells Python to go through the inventory list one item at a time, temporarily storing each one inside the variable item.

On the first pass, item becomes "sword", and the f-string prints You have a sword.

This repeats automatically for every item in the list "shield", then "potion" without us ever needing to know exactly how many items are in the list.

You can even combine range(), len(), and a list to loop using numbered index positions, which is useful when you need the position itself, not just the value:

```
inventory = ["sword", "shield", "potion"]

for i in range(len(inventory)):
    print(f"Slot {i}: {inventory[i]}")
```

□ Here's What's Happening:

len(inventory) gives us 3, so range(len(inventory)) produces 0, 1, 2 one index for every item in the list.

for i in range(len(inventory)): loops through those index numbers, storing each one in i.

inventory[i] then looks up whichever item lives at that specific index, letting us display both the slot number and its contents together.

Tuples: Lists That Can't Change

A tuple looks a lot like a list, but is written with parentheses instead of square brackets, and this is the key difference it cannot be changed once created. We say tuples are immutable.

```
starting_position = (0, -250)
print(starting_position)
print(starting_position[0])
print(starting_position[1])
```

□ Here's What's Happening:

starting_position = (0, -250) creates a tuple holding two values, an x-coordinate and a y-coordinate.

Just like a list, we can access individual items using square brackets and an index: starting_position[0] gives us 0, and starting_position[1] gives us -250.

Unlike a list, though, trying to run starting_position[0] = 100 would cause an error, because tuples cannot be modified after they're created.

□ Tip:

Use a tuple whenever you have a small, fixed group of values that should never change, like a starting coordinate or an RGB color. Use a list whenever the collection needs to grow, shrink, or be edited over time, like an inventory.

Dictionaries

A dictionary stores information in key-value pairs, like a real dictionary matches a word to its definition. Instead of using numbered positions like a list, you look things up by a meaningful key.

```
player = {"name": "Ada", "health": 100, "level": 1}
print(player["name"])
print(player["health"])
```

□ Here's What's Happening:

`player = {...}` creates a dictionary, using curly braces `{ }` instead of the square brackets a list uses.

Inside, each entry is written as `key: value`, separated by commas. `"name"` is a key that maps to the value `"Ada"`; `"health"` maps to `100`; `"level"` maps to `1`.

`player["name"]` looks up the value stored under the key `"name"`, giving us `"Ada"` notice we use the meaningful key instead of a numbered index.

`player["health"]` looks up the value under the `"health"` key, giving us `100`.

```
player = {"name": "Ada", "health": 100}
player["health"] = 80
print(player)
```

□ Here's What's Happening:

`player["health"] = 80` finds the `"health"` key and replaces its current value with `80` exactly the same pattern we used to update a regular variable, just aimed at one entry inside the dictionary.

`print(player)` shows the whole updated dictionary: `{'name': 'Ada', 'health': 80}`.

You can also add a brand-new key to a dictionary just by assigning a value to it:

```
player = {"name": "Ada", "health": 100}
player["level"] = 1
print(player)
```

□ Here's What's Happening:

Since `"level"` doesn't exist yet in this dictionary, `player["level"] = 1` creates a new key-value pair rather than updating an existing one.

`print(player)` now shows all three entries: `{'name': 'Ada', 'health': 100, 'level': 1}`.

Looping over a dictionary is slightly different from looping over a list. The `.items()` method gives you both the key and the value together on each pass:

```
player = {"name": "Ada", "health": 100, "level": 1}

for key, value in player.items():
    print(f"{key}: {value}")
```

□ Here's What's Happening:

`player.items()` produces a sequence of key-value pairs from the dictionary.

`for key, value in ...:` unpacks each pair into two separate variables at once `key` holds the label (like "name"), and `value` holds what's stored there (like "Ada").

This loop runs once for every entry in the dictionary, printing each key and its matching value on its own line.

□ Tip:

Use a list when order matters and items are similar (like a row of enemies). Use a dictionary when you want to label pieces of related information (like all the stats belonging to one player). Use a tuple when a small group of values should never change.

Importing Modules

We already imported the `random` module in Week 2. Now that you know about functions, you can think of a module as a file full of other people's ready-made, reusable functions, which you can bring into your own program with the `import` keyword.

```
import math

print(math.sqrt(16))
print(math.pi)
```

□ Here's What's Happening:

`import math` loads Python's built-in `math` module, which is packed with mathematical functions and constants.

`math.sqrt(16)` calls the `sqrt` (square root) function that lives inside the `math` module, passing it 16.

`print(...)` displays the result, 4.0 the square root of 16.

`math.pi` isn't a function at all, but a constant value stored inside the module accessing it displays Python's built-in, highly precise value for `pi`.

□ Mini Challenge: Party Roster

Create a list called `party` containing the names of three adventurers.

Create a dictionary called `hero` containing keys for name, health, and weapon.

Use a for loop to print a greeting for every member of `party`.

Print a single sentence describing your hero, using values pulled out of the `hero` dictionary.

Create a tuple called `starting_stats` holding a starting strength and starting speed value, and explain in a comment why a tuple makes sense here.

Graphics: The Turtle Module

So far, everything we've built has lived inside the text-only Run window. It's time to open an actual graphics window and draw something! Python includes a built-in module called `turtle`, named after a friendly little robot turtle that drags a pen behind it as it moves.

```
import turtle

window = turtle.Screen()
my_turtle = turtle.Turtle()

my_turtle.forward(100)

window.exitonclick()
```

□ Here's What's Happening:

`import turtle` loads the `turtle` module, giving us access to its graphics tools.

`turtle.Screen()` creates the actual graphics window that will pop up on your screen, and we store it in a variable called `window` so we can control it later.

`turtle.Turtle()` creates our turtle the little arrow-shaped cursor that will do our drawing and we store it in a variable called `my_turtle`.

`my_turtle.forward(100)` tells our turtle to move forward 100 pixels, drawing a straight line behind it as it goes.

`window.exitonclick()` keeps the graphics window open until you click inside it, giving you time to admire your drawing instead of it vanishing instantly.

□ Tip:

Every turtle command follows the same pattern: the name of your turtle variable, a dot, then the action, like `my_turtle.forward(100)` or `my_turtle.right(90)`. Get comfortable with this pattern you'll use it constantly.

This idea of "a variable, a dot, then an action" is worth pausing on. Whenever you see `variable.something()`, it means: "take this specific object, and run one of the special actions it knows how to do." We'll see this exact pattern again constantly, including throughout our game next week.

Drawing Shapes, Colors, Speed, Coordinates & Pen Control

Now let's give our turtle some real commands to work with. Here's a small toolbox of the most useful turtle actions:

- `forward(distance)` / `backward(distance)` moves the turtle in a straight line.
- `right(angle)` / `left(angle)` rotates the turtle by a number of degrees, without moving it.
- `penup()` / `pendown()` lifts the pen so the turtle can move without drawing, or lowers it so it draws again.
- `color("colorname")` changes the color of the line the turtle draws.
- `speed(number)` sets how fast the turtle moves, from 1 (slowest) to 10 (fastest), or 0 for instant.

Let's combine several of these to draw a square:

```
import turtle

window = turtle.Screen()
my_turtle = turtle.Turtle()
my_turtle.color("blue")
my_turtle.speed(3)

for side in range(4):
    my_turtle.forward(100)
    my_turtle.right(90)

window.exitonclick()
```

□ Here's What's Happening:

`my_turtle.color("blue")` sets every line drawn from this point forward to be blue.

`my_turtle.speed(3)` sets a slow-ish, easy-to-watch drawing speed.

`for side in range(4):` repeats the indented block exactly four times perfect for a shape with four equal sides.

Each pass through the loop, `my_turtle.forward(100)` draws one side of the square, and `my_turtle.right(90)` rotates the turtle a quarter-turn to prepare for the next side.

After all four passes, the turtle has drawn all four sides and rotated a full 360 degrees, ending up facing exactly the direction it started that's how we know the square is complete.

□ **Tip:**

Try changing `range(4)` to `range(6)` and 90 to 60 to draw a hexagon instead. Experimenting like this is one of the best ways to truly understand how the turtle module works.

Common Mistakes to Avoid:

- Forgetting `window.exitonclick()` (or another way of keeping the window open), causing the graphics window to flash open and immediately close.
- Using the wrong turn angle for a shape remember, the angles for a regular shape's turns should always add up to 360 degrees in total.

Turtle also has a built-in shortcut for perfect circles, without needing a loop at all:

```
my_turtle.color("green")
my_turtle.circle(50)
```

□ **Here's What's Happening:**

`my_turtle.circle(50)` draws a complete circle with a radius of 50 pixels, all in a single command.

Unlike our square, we didn't need a loop here, since `circle()` already knows how to draw the entire curved shape by itself.

You can combine a loop with small turns to build more elaborate patterns, like a simple spiral:

```
my_turtle.color("purple")
my_turtle.speed(0)

for step in range(50):
    my_turtle.forward(step * 2)
    my_turtle.right(91)
```

□ **Here's What's Happening:**

`my_turtle.speed(0)` sets the turtle's fastest possible speed, since this pattern involves many small movements and would look sluggish otherwise.

`for step in range(50):` repeats 50 times, using `step` as a counter that increases by 1 on every pass (0, 1, 2, and so on).

`my_turtle.forward(step * 2)` moves the turtle a distance that grows a little larger every single pass, since `step` keeps increasing.

`my_turtle.right(91)`, a turn just slightly more than a quarter-circle, combined with the ever-growing forward distance, causes the whole pattern to spiral gradually outward.

The Coordinate System

Every turtle graphics window uses a coordinate system to describe exact positions, just like a map. The very center of the window is the point (0, 0).

- The x-coordinate tells you how far left (negative) or right (positive) a point is from the center.
- The y-coordinate tells you how far down (negative) or up (positive) a point is from the center.
- `goto(x, y)` instantly moves the turtle to an exact coordinate on the screen.

```
import turtle

window = turtle.Screen()
my_turtle = turtle.Turtle()

my_turtle.penup()
my_turtle.goto(-100, 50)
my_turtle.pendown()
my_turtle.goto(100, 50)

window.exitonclick()
```

□ Here's What's Happening:

`my_turtle.penup()` lifts the pen, so the next movement won't draw a line this is essential whenever you want to reposition the turtle without leaving a mark.

`my_turtle.goto(-100, 50)` instantly teleports the turtle to the point 100 pixels left of center and 50 pixels above center.

`my_turtle.pendown()` lowers the pen again, so the turtle will resume drawing.

`my_turtle.goto(100, 50)` moves the turtle to a point 100 pixels right of center, at the same height. Because the pen is down, this movement draws a straight horizontal line between the two points.

You can also read a turtle's current position at any moment using `xcor()` and `ycor()`:

```
print(my_turtle.xcor())
print(my_turtle.ycor())
```

□ Here's What's Happening:

`my_turtle.xcor()` returns the turtle's current horizontal position as a number.

`my_turtle.ycor()` returns the turtle's current vertical position as a number.

These two tools will become extremely important next week, when we need to constantly check exactly where our game objects are.

□ Tip:

Understanding (x, y) coordinates now will make Week 4 dramatically easier, since every game object your player, your enemies, your bullets will need its own exact position on screen.

□ Mini Challenge: Draw a Simple House

Use `forward()` and `right()/left()` to draw a square for the base of the house.

Use `penup()`, `goto()`, and `pendown()` to reposition the turtle above the square.

Draw a triangle on top of the square to act as the roof.

Bonus: use `color()` to make the roof a different color from the base, and add a `circle()` for the sun in the sky.

Sorting Lists

Keeping a list in order like a leaderboard sorted from highest to lowest score is such a common need that Python includes a built-in way to do it.

```
scores = [42, 17, 89, 3, 65]
scores.sort()
print(scores)
```

```
scores.sort(reverse=True)
print(scores)
```

□ Here's What's Happening:

`scores.sort()` rearranges the items inside the scores list itself, from smallest to largest, permanently changing the list's order.

The first print shows `[3, 17, 42, 65, 89]` the same five numbers, now in ascending order.

`scores.sort(reverse=True)` sorts the list again, but this time from largest to smallest, which is exactly what a high-score leaderboard needs.

The second print shows `[89, 65, 42, 17, 3]`.

□ Tip:

`sort()` changes the original list directly and doesn't return a new one so writing `scores = scores.sort()` would actually replace your list with `None`! Just call `scores.sort()` on its own line.

A Common Gotcha: Copying Lists

Lists behave a little differently from simple variables like integers or strings when you try to copy them. This trips up even experienced programmers, so it's worth seeing early.

```
original = ["sword", "shield"]
copy = original
copy.append("potion")

print(original)
print(copy)
```

□ Here's What's Happening:

`copy = original` might look like it creates a brand-new, separate list, but it doesn't it simply gives the exact same list a second name to go by. Both `original` and `copy` point to the very same list in the computer's memory.

`copy.append("potion")` adds "potion" to that shared list.

Because there was never actually a second, separate list, printing `original` also shows the new "potion" item, even though we only ever changed `copy`.

To create a truly independent copy, you'd use `copy = original.copy()` or `copy = list(original)` instead, either of which creates a brand-new list with the same starting contents.

□ Tip:

This same behavior applies to dictionaries too. Whenever you want a completely separate, independent copy of a list or dictionary, use `.copy()` rather than a plain equal sign.

More Turtle Patterns: Looping Shapes

Let's combine a for loop with turtle graphics one more time, to draw a whole ring of shapes a nice preview of the kind of visual polish you'll add to your finished game in Week 5.

```
import turtle

window = turtle.Screen()
my_turtle = turtle.Turtle()
my_turtle.speed(0)

colors = ["red", "orange", "yellow", "green", "blue", "purple"]
```

```
for i in range(6):
    my_turtle.color(colors[i])
    my_turtle.circle(60)
    my_turtle.right(60)

window.exitonclick()
```

□ Here's What's Happening:

`colors = [...]` is a list of six color names another small preview of lists, which you now understand fully after this week's earlier section.

`for i in range(6):` repeats six times, using `i` as a counter from 0 through 5.

`my_turtle.color(colors[i])` uses that counter as an index into the `colors` list, picking a different color on each pass through the loop.

`my_turtle.circle(60)` draws a circle, and `my_turtle.right(60)` rotates the turtle slightly before drawing the next one, so the six circles fan out in a ring instead of stacking directly on top of each other.

□ Mini Challenge: Design Your Own Pattern

Modify the ring-of-circles example to use 8 shapes instead of 6, adjusting the rotation angle so they still form a complete ring.

Replace `circle(60)` with a small square-drawing loop (reusing your Week 3 square code) to create a ring of squares instead.

Week 3 Recap

- Functions, created with `def`, package up reusable blocks of code, using parameters as their inputs.
- `return` hands a value back out of a function, so it can be stored or used elsewhere.
- Lists (`[]`) store ordered, changeable collections of values, accessed using zero-based indexes.
- Tuples (`()`) look similar to lists but cannot be changed once created perfect for fixed values like coordinates.
- Dictionaries (`{}`) store labeled key-value pairs, accessed using meaningful keys, and looped over using `.items()`.
- `import` brings in extra tools from a module, like `math` or `turtle`.
- The `turtle` module draws graphics using commands like `forward()`, `right()`, `circle()`, `color()`, `penup()`, and `pendown()`.
- Every position on a turtle screen is described using (x, y) coordinates, with (0, 0) at the exact center.

Check Your Understanding

31. What is the difference between a parameter and an argument?
32. Why would a function use return instead of just using print()?
33. What is stored at inventory[0] versus inventory[-1] in a three-item list?
34. When would you choose a dictionary instead of a list to store information?
When would you choose a tuple instead of a list?
35. What does penup() do, and why is it important before calling goto()?

Fantastic work! You now know how to build your own functions, organize data with lists, tuples, and dictionaries, and draw shapes on screen using coordinates. Next week, we combine every single one of these skills to build the actual game.

WEEK 4

Game Mechanics & Physics

Building Alien Dodge your first real, playable game

The Core Game Loop

This week, every skill from the last three weeks comes together. We're going to build a real game from scratch: Alien Dodge. You'll control a little ship at the bottom of the screen, dodge falling aliens, and rack up points for every second you survive.

Every video game, from the simplest mobile game to the biggest console blockbuster, is built around one core idea: the game loop. A game loop is a block of code that repeats over and over, extremely fast, and on every single repeat it does three things, in this order:

36. Check for anything new like a key the player just pressed.
37. Update the game move objects, check for collisions, update the score.
38. Redraw the screen show the player the new positions of everything.

This all happens dozens of times per second, which is why movement in games looks smooth rather than jumpy. Let's start building.

Planning Our Game

Before writing any code, let's write our algorithm in plain English, just like we practiced in Week 1:

39. Create a window, a player ship, and a falling alien.
40. Let the player move their ship left and right using the arrow keys.
41. Make the alien fall continuously, resetting to a new random position at the top whenever it reaches the bottom.
42. Check whether the falling alien has touched the player's ship.
43. Keep score for every moment the player survives, and track a limited number of lives.
44. End the game with a win message if the score reaches a target, or a lose message if lives run out.

Every section below builds one piece of this plan, in order.

Game Objects

First, we need to set up our window and create our player's ship as a turtle object.

```
import turtle

window = turtle.Screen()
window.title("Alien Dodge")
window.bgcolor("black")
window.setup(width=600, height=600)
window.tracer(0)

player = turtle.Turtle()
player.shape("triangle")
player.color("cyan")
player.penup()
player.goto(0, -250)
```

□ Here's What's Happening:

`window.title("Alien Dodge")` sets the text that appears at the top of our game window. `window.bgcolor("black")` paints the background black, which will make our bright player ship and aliens stand out just like a real night-sky arcade game.

`window.setup(width=600, height=600)` fixes the size of our game window in pixels, so we always know exactly how big our play area is.

`window.tracer(0)` turns off turtle's automatic screen updates. This is essential for smooth games instead of redrawing the screen after every tiny movement, we will decide exactly when to redraw it ourselves, which is much faster.

`player = turtle.Turtle()` creates our player object. `player.shape("triangle")` gives it a simple triangle shape (a classic "spaceship" look), and `player.color("cyan")` makes it a bright, easy-to-see blue.

`player.penup()` makes sure our player doesn't leave a drawn line trailing behind it as it moves we want it to glide, not draw.

`player.goto(0, -250)` places the player near the bottom-center of the screen, using the coordinate system you learned in Week 3.

□ Tip:

Notice how every single one of these lines uses ideas from earlier weeks: variables (Week 1), the turtle module and coordinates (Week 3). Game development is really just combining familiar building blocks in new ways.

Keyboard Controls

Next, we need to let the player actually control their ship using the left and right arrow keys.

```
def move_left():
    new_x = player.xcor() - 20
    if new_x < -280:
        new_x = -280
    player.setx(new_x)

def move_right():
    new_x = player.xcor() + 20
    if new_x > 280:
        new_x = 280
    player.setx(new_x)

window.listen()
window.onkeypress(move_left, "Left")
window.onkeypress(move_right, "Right")
```

□ Here's What's Happening:

We define a function `move_left()`. Inside it, `player.xcor()` asks the player turtle for its current x-coordinate (how far left or right it currently is), and we subtract 20 to move it a bit further left.

The `if new_x < -280:` line stops our ship from sliding off the left edge of the 600-pixel-wide window it's a simple boundary check, capping the value at -280 instead of letting it go further.

`player.setx(new_x)` moves the player horizontally to this new x position, without touching its y position (its height) at all.

`move_right()` works exactly the same way, just adding 20 instead of subtracting, and capping the value at 280 to protect the right edge.

`window.listen()` tells the graphics window, "start paying attention to keyboard presses." Without this line, none of our key bindings would work.

`window.onkeypress(move_left, "Left")` connects the Left arrow key to our `move_left` function. This is called event handling instead of constantly checking "is the key pressed yet?", we tell Python, "whenever this specific event happens, run this specific function," and Python takes care of watching for it.

□ Tip:

Try running just this much of the game so far (you'll need `window.update()` in a loop to actually see anything that's coming up next!) and pressing the arrow keys to get a feel for event handling.

Some players prefer the A and D keys over the arrow keys. Since `onkeypress` lets us bind more than one key to the same function, adding both is easy:

```
window.onkeypress(move_left, "a")
window.onkeypress(move_right, "d")
```

□ Here's What's Happening:

These two extra lines bind the lowercase "a" and "d" keys to the exact same `move_left` and `move_right` functions we already wrote.

Because event handling just connects a key to a function, we can connect as many different keys as we like to the very same function, without duplicating any code.

Now the player can use either the arrow keys or A and D, whichever feels more comfortable a small touch of accessibility that many real games include.

Event Handling and the Loop Itself

Now let's write the actual game loop function that ties everything together and keeps re-running itself.

```
def game_loop():
    window.update()
    window.ontimer(game_loop, 33)

game_loop()
window.mainloop()
```

□ Here's What's Happening:

`window.update()` is the "redraw the screen" step it takes every change we've made (like the player moving) and actually displays it. Because we turned off automatic updates earlier with `tracer(0)`, this is the only moment the screen visually refreshes.

`window.ontimer(game_loop, 33)` is the secret to the whole loop: it tells Python, "wait 33 milliseconds, and then call `game_loop()` again." Because `game_loop()` calls itself again through `ontimer`, it keeps repeating forever roughly 30 times every second, which feels smooth to the human eye.

The line `game_loop()` outside the function is what starts the very first cycle of the loop.

`window.mainloop()` keeps the entire graphics window alive and responsive, patiently waiting for events like key presses and timers to fire. Without it, the window would open and close instantly.

□ **Tip:**

33 milliseconds might look like an odd number, but 1000 milliseconds (one second) divided by 33 is about 30 so this loop runs roughly 30 times per second, a smooth and very common speed for simple games.

This number how many times the loop runs per second is called the frame rate, usually measured in frames per second (FPS). A higher frame rate (a smaller number of milliseconds in `ontimer`) makes movement look smoother, but asks the computer to do more work more often.

- `window.ontimer(game_loop, 16)` would run roughly 60 times per second a very smooth, common frame rate for modern games.
- `window.ontimer(game_loop, 100)` would run only 10 times per second noticeably choppy, but still playable, and useful for testing on a slower computer.

Common Mistakes to Avoid:

- Forgetting `window.tracer(0)` at setup, or forgetting `window.update()` inside the loop either mistake will make your game either flicker badly or never visually update at all.
- Placing `window.mainloop()` before `game_loop()` is first called Python needs to start the loop running before it settles into just waiting for events.
- Calling `window.ontimer(game_loop, 33)` with the parentheses, like `game_loop()`, instead of just the function's name, `game_loop` this is a subtle but important difference: we want to hand `ontimer` the function itself to call later, not run it immediately and hand over its result.

□ **Mini Challenge: Test Your Ship**

Combine the three code blocks above into a single file.

Run it, and use the Left and Right arrow keys (and the A/D keys, if you added them) to move your triangle ship back and forth.

Confirm your ship stops at the edges of the window instead of sliding off screen.

Try changing 33 to 100 in `window.ontimer()`, and notice how much choppy the (currently invisible, since nothing else moves yet) frame rate would feel once we add moving aliens.

Collision Detection: Did We Hit the Alien?

A ship that just slides left and right isn't much of a game yet. Let's add falling aliens for our player to dodge, and teach Python how to notice when two objects touch this is called collision detection.

Random Movements

First, let's create a single alien that starts at a random position along the top of the screen and falls downward.

```
import random

alien = turtle.Turtle()
alien.shape("circle")
alien.color("red")
alien.penup()
alien.goto(random.randint(-280, 280), 280)
```

□ Here's What's Happening:

`alien = turtle.Turtle()` creates a second, completely separate turtle object to represent our falling alien.

`alien.shape("circle")` and `alien.color("red")` give it a distinct round, red appearance, so it's easy to tell apart from our cyan triangle ship.

`random.randint(-280, 280)` picks a random whole number somewhere between -280 and 280 a random horizontal spot within our play area.

`alien.goto(...)` places the alien at that random x position, right near the top of the screen (`y = 280`), so every time we run the game, the alien starts somewhere slightly different just like the random module we learned in Week 2.

Now let's make the alien actually fall, and reappear at a new random position once it reaches the bottom. This logic belongs inside our game loop, since it needs to happen on every single cycle.

```
def game_loop():
    alien.sety(alien.ycor() - 5)

    if alien.ycor() < -300:
        alien.goto(random.randint(-280, 280), 280)

    window.update()
    window.ontimer(game_loop, 33)
```

□ Here's What's Happening:

`alien.sety(alien.ycor() - 5)` reads the alien's current height, subtracts 5, and moves it down slightly. Since this line runs about 30 times per second inside our loop, the alien appears to fall smoothly.

`if alien.ycor() < -300:` checks whether the alien has fallen past the bottom of the visible screen.

If it has, `alien.goto(random.randint(-280, 280), 280)` resets it to a brand-new random position back at the top creating an endless, ever-changing stream of falling aliens using a single reusable turtle object.

□ **Tip:**

This technique resetting an object instead of creating a brand-new one every time is called recycling or object pooling, and it's a common trick in real games to keep performance smooth.

Detecting a Collision

Now for the exciting part: figuring out whether the alien has touched the player. Turtle objects have a handy built-in tool for exactly this: the `distance()` method, which calculates the distance between two turtles.

```
def game_loop():
    alien.sety(alien.ycor() - 5)

    if alien.ycor() < -300:
        alien.goto(random.randint(-280, 280), 280)

    if alien.distance(player) < 20:
        print("Ouch! You got hit!")
        alien.goto(random.randint(-280, 280), 280)

    window.update()
    window.ontimer(game_loop, 33)
```

□ **Here's What's Happening:**

`alien.distance(player)` calculates the straight-line distance, in pixels, between the alien turtle and the player turtle, based on both of their current coordinates.

`if alien.distance(player) < 20:` checks whether that distance has shrunk small enough to count as a hit. We're using 20 pixels as a reasonable collision distance for our small shapes feel free to experiment with this number.

When a collision is detected, we print a message, and then we send the alien back to a new random position at the top, so the game can continue rather than getting stuck with the alien overlapping the player forever.

□ **Tip:**

The number 20 in our collision check is sometimes called a hitbox size. Make it too small, and collisions will feel unfair, like the game missed an obvious hit. Make it too big, and the game will feel unfair in the opposite way, punishing near-misses.

Tweaking this number by playtesting is a real part of professional game design!

Common Mistakes to Avoid:

- Checking the collision distance only once, outside of the loop, instead of on every single pass collisions must be checked continuously as objects move.
- Choosing a hitbox number that's too large for your shapes, causing the game to register a hit even when the shapes clearly haven't touched on screen.
- Forgetting that `distance()` works between any two turtles, not just the player and a single alien you'll use this exact same tool for every additional enemy you add.

Adding a Second Enemy Type

One alien falling in a straight line gets predictable fast. Let's add a second kind of enemy a faster, smaller "asteroid" to keep the player on their toes.

```
asteroid = turtle.Turtle()  
asteroid.shape("triangle")  
asteroid.color("orange")  
asteroid.shapesize(0.5, 0.5)  
asteroid.penup()  
asteroid.goto(random.randint(-280, 280), 280)
```

□ **Here's What's Happening:**

`asteroid = turtle.Turtle()` creates yet another independent turtle object, completely separate from both player and alien.

`asteroid.shapesize(0.5, 0.5)` shrinks the default shape down to half its normal size, giving us a visually distinct, smaller enemy.

The rest of the setup mirrors exactly what we did for alien, just with a different color and shape, reusing a pattern we already understand instead of inventing something new.

Now we add its falling behavior and its own collision check into the game loop, right alongside the alien's:

```
def game_loop():  
    alien.sety(alien.ycor() - 5)  
    asteroid.sety(asteroid.ycor() - 9)
```

```

if alien.ycor() < -300:
    alien.goto(random.randint(-280, 280), 280)

if asteroid.ycor() < -300:
    asteroid.goto(random.randint(-280, 280), 280)

if alien.distance(player) < 20 or asteroid.distance(player)
< 15:
    alien.goto(random.randint(-280, 280), 280)
    asteroid.goto(random.randint(-280, 280), 280)

window.update()
window.ontimer(game_loop, 33)

```

□ Here's What's Happening:

`asteroid.sety(asteroid.ycor() - 9)` makes the asteroid fall faster than the alien (9 pixels per pass instead of 5), giving it a different, more dangerous feel.

Each enemy gets its own independent boundary check, resetting only that specific enemy once it falls off the bottom of the screen.

`if alien.distance(player) < 20 or asteroid.distance(player) < 15`: uses the `or` logical operator from Week 2 to check both enemies in a single condition. If either one is close enough to the player, the whole condition becomes `True`.

Notice the asteroid uses a smaller hitbox distance (15 instead of 20), matching its smaller visual size. Hitboxes should generally match how big a shape actually looks on screen.

□ Tip:

Adding a second enemy didn't require any brand-new ideas—just the same patterns (a turtle, a fall, a boundary reset, a collision check) applied a second time. This is exactly how real games grow: by reusing solid patterns, not reinventing them.

□ Mini Challenge: Double Trouble

Add the asteroid enemy to your own game, following the code above.

Playtest with both enemies falling at once: does it feel too easy, too hard, or just right?

Try adding a third enemy type of your own, with its own shape, color, speed, and hitbox size.

Scoring and On-Screen Text

A game needs a way to tell the player how they're doing. Let's add a score that increases the longer you survive, a limited number of lives, a simple timer, and clear win and lose conditions.

Displaying On-Screen Text

To draw text in a turtle window, we use another turtle purely as a "pen" for writing, rather than for moving around visibly.

```
scoreboard = turtle.Turtle()
scoreboard.hideturtle()
scoreboard.penup()
scoreboard.color("white")
scoreboard.goto(-280, 260)

score = 0
lives = 3

scoreboard.write(f"Score: {score}  Lives: {lives}",
font=("Arial", 16, "normal"))
```

□ Here's What's Happening:

`scoreboard = turtle.Turtle()` creates yet another turtle, but this one's job is only to display text, never to move around as a visible character.

`scoreboard.hideturtle()` hides the little triangular turtle icon itself, leaving only the text it writes.

`scoreboard.goto(-280, 260)` positions it in the top-left corner of the screen, a common spot for game information.

`score = 0` and `lives = 3` are the two variables we'll track throughout the game `score` starts at zero, and the player starts with 3 lives.

`scoreboard.write(...)` draws text at the scoreboard's current position. The f-string builds our message using the current score and lives, and `font=("Arial", 16, "normal")` sets the text's font, size, and style.

Since score and lives will change constantly, we need to clear the old text and rewrite it every single loop. Let's build this into our full game loop:

```
def game_loop():
    global score, lives

    alien.sety(alien.ycor() - 5)
    score += 1

    if alien.ycor() < -300:
        alien.goto(random.randint(-280, 280), 280)
```

```

    if alien.distance(player) < 20:
        lives -= 1
        alien.goto(random.randint(-280, 280), 280)

    scoreboard.clear()
    scoreboard.write(f"Score: {score}  Lives: {lives}",
font=("Arial", 16, "normal"))

    if lives <= 0:
        scoreboard.goto(-70, 0)
        scoreboard.write("GAME OVER", font=("Arial", 28,
"bold"))
        return

    window.update()
    window.ontimer(game_loop, 33)

```

□ Here's What's Happening:

global score, lives tells Python that when we change score or lives inside this function, we mean the same variables we created outside the function, not brand-new local copies. Without this line, our changes to score and lives would disappear the moment the function finished.

score += 1 adds one point to the score on every single pass through the loop since the loop runs about 30 times a second, the player's score climbs quickly the longer they survive, rewarding survival time.

When a collision happens, lives -= 1 removes one life instead of just printing a message like before.

scoreboard.clear() erases the previously written text so the old numbers don't overlap with the new ones always clear before rewriting turtle text.

scoreboard.write(...) then draws the updated score and lives count using the current values.

if lives <= 0: is our lose condition. Once the player runs out of lives, we move the scoreboard turtle to the center of the screen, write GAME OVER in large bold text, and then return, which exits the game_loop function completely and since game_loop no longer calls window.ontimer to schedule itself again, the loop simply stops running.

□ Tip:

Notice that return does two jobs here: it ends the function's execution right away, and because nothing schedules game_loop again afterward, it's also how we cleanly stop the entire game loop. This is a common and elegant pattern in simple games.

Why does forgetting global matter so much? Let's see what would happen without it:

```
score = 0

def broken_increment():
    score = score + 1 # this line will crash!

broken_increment()
```

❑ Here's What's Happening:

Without the word `global`, Python assumes that any variable you assign to inside a function (like `score = ...` here) is a brand-new local variable, separate from the `score` outside the function.

But `score = score + 1` tries to read the value of this new local `score` before it has ever been given one, causing an `UnboundLocalError`.

Adding `global score` as the very first line inside the function fixes this completely, by telling Python clearly, "I mean the outer `score`, not a new one."

Displaying Lives as Icons

Printing "Lives: 3" works, but many games show lives as little icons instead hearts, ships, or stars. Let's build a simple version of that using a string of a repeated character.

```
lives = 3
life_icons = "♥" * lives
print(life_icons)
```

❑ Here's What's Happening:

`"♥" * lives` multiplies a string by a number a neat trick in Python where repeating a string with `*` works similarly to repeating a number.

Since `lives` is 3, this produces the heart character repeated three times in a row: a small string showing three hearts.

You could use this `life_icons` string anywhere you'd otherwise show the plain number, like inside our `scoreboard.write(...)` f-string, for a more visual health display.

❑ Tip:

Multiplying a string by a number (like `"*" * 5`) is a fun shortcut worth remembering it works for building simple visual bars, borders, and repeated icons like this one.

Timers and Win Conditions

Right now, the only way to "win" is to keep dodging forever, which isn't a very satisfying goal. Let's add a win condition: reach a target score, like 500 points, before running out of lives.

```
if score >= 500:
    scoreboard.goto(-100, 0)
    scoreboard.write("YOU WIN!", font=("Arial", 28, "bold"))
    return
```

□ Here's What's Happening:

This check works exactly like our lose condition, but it looks for the opposite outcome: the score reaching a target number instead of lives running out.

Just like before, writing the win message and then calling return stops the game loop from scheduling itself again, ending the game cleanly the moment the player wins.

You would place this check right alongside the lives ≤ 0 check inside game_loop, so both the win and lose conditions get checked on every single loop.

You can also add a visible countdown timer, using the same "another turtle as a pen" technique, combined with a variable that decreases over time:

```
time_left = 30 # seconds

# Inside game_loop, roughly once per second:
time_left -= 1
if time_left <= 0:
    scoreboard.goto(-100, 0)
    scoreboard.write("TIME'S UP!", font=("Arial", 28, "bold"))
    return
```

□ Here's What's Happening:

time_left = 30 stores how many seconds remain, set outside the loop so it only gets created once.

Since our loop runs about 30 times per second, you would only want to subtract from time_left roughly once every 30 passes, not on every single one otherwise your 30-second timer would run out almost instantly. A common way to handle this is with a separate counter variable that resets every 30 loops.

if time_left ≤ 0 : works exactly like our other win/lose checks, ending the game once time runs out.

□ Mini Challenge: Finish the Game

Add the score ≥ 500 win check into your full game_loop function, right next to the lives ≤ 0 lose check.

Playtest your game several times. Does 500 points feel too easy or too hard to reach? Adjust the target score, the falling speed, or the number of starting lives until it feels fun.

Try adding the life_icons heart display to your scoreboard instead of the plain number.

Adding a Start Screen

Right now, our game throws the player straight into the action the instant it runs. Many real games instead show a title or start screen first, waiting for the player to signal they're ready. Let's add a simple version.

```
game_started = False

def start_game():
    global game_started
    game_started = True
    title_text.clear()

title_text = turtle.Turtle()
title_text.hideturtle()
title_text.penup()
title_text.color("white")
title_text.goto(-150, 0)
title_text.write("ALIEN DODGE\nPress SPACE to start",
font=("Arial", 20, "bold"))

window.onkeypress(start_game, "space")
```

□ Here's What's Happening:

`game_started = False` is a boolean flag, similar to the `is_game_over` pattern from Week 2, tracking whether the player has begun playing yet.

`start_game()` is a new function we bind to the space bar. When called, it flips `game_started` to `True` using `global`, and clears away the title text.

`title_text` is yet another "invisible writing turtle," following the same pattern as our scoreboard, used purely to display the game's title and instructions before play begins.

`window.onkeypress(start_game, "space")` connects the space bar to our new function, using the same event handling pattern we used for the arrow keys.

The last step is to make our `game_loop` function check this flag before actually running any game logic:

```
def game_loop():
    global score, lives, fall_speed

    if not game_started:
        window.update()
```

```
window.ontimer(game_loop, 33)
return
```

```
# ...rest of the game logic goes here, exactly as before
```

□ Here's What's Happening:

if not game_started: checks whether the player has not yet pressed space, using the not operator from Week 2.

While that's True, the function only updates the screen and reschedules itself, then immediately returns completely skipping the falling alien, collision checks, and scoring below it.

The moment start_game() flips game_started to True, this if condition becomes False on the very next loop, and the rest of the function runs normally from then on.

□ Tip:

This "skip everything and return early if a condition isn't met yet" pattern is extremely common in real games, for start screens, pause menus, and cutscenes alike.

Real-World Connection: Vertical Movement

Our ship only moves left and right, but the exact same setx() pattern works for up-and-down movement too, using sety() instead useful if you'd like to expand your game further.

```
def move_up():
    new_y = player.ycor() + 20
    if new_y > 250:
        new_y = 250
    player.sety(new_y)
```

```
window.onkeypress(move_up, "Up")
```

□ Here's What's Happening:

This function mirrors move_left() and move_right() exactly, just working with ycor() and sety() instead of xcor() and setx(), and capping the value at 250 instead of 280 to fit our window's height.

Adding a matching move_down() function, plus binding both to the Up and Down arrow keys, would let your ship move freely in all four directions a small change that opens up a lot of new gameplay possibilities.

□ Mini Challenge: Add a Start Screen

Add the title_text and start_game() code to your own game.

Update your `game_loop` function with the `if not game_started`: early-return check.

Optional: add full four-directional movement using the `move_up()` pattern above.

Week 4 Recap

- The game loop is a repeating cycle: check input, update the game, redraw the screen.
- `window.tracer(0)` plus manual `window.update()` calls give you full control over when the screen refreshes, for smooth animation.
- `window.ontimer(function_name, milliseconds)` is how a game loop schedules itself to repeat, and controls the frame rate.
- `window.listen()` and `window.onkeypress()` connect keyboard presses to your own functions event handling and a single function can be bound to multiple keys.
- `distance()` checks how far apart two turtles are, which is how we detect collisions.
- Multiple enemy types reuse the same patterns: a turtle, a fall, a boundary reset, and a collision check.
- `global` lets a function change a variable that was created outside of it.
- Win and lose conditions are just `if` statements checking a score or a lives count, paired with a `return` that stops the loop.

Check Your Understanding

45. What are the three steps every game loop repeats, in order?
46. Why do we call `window.tracer(0)` before the loop starts, instead of leaving turtle's automatic updates on?
47. What does `alien.distance(player)` calculate, and how do we use that number to detect a collision?
48. What would happen if you removed `global score, lives` from the top of `game_loop`?
49. Why does `return`, by itself, effectively stop our entire game loop from continuing?
50. How did adding the asteroid enemy reuse patterns from the alien, instead of requiring brand-new ideas?

You did it you just built a real, working, playable video game with a moving player, falling enemies, collision detection, a live scoreboard, and both a win and a lose condition. Next week, we'll polish this game until it shines, and get you ready to show it off.

WEEK 5

Final Project & Game Polish

Testing, polishing, and presenting Alien Dodge

Testing

You have a working game. Now it's time to put on your tester hat. Testing means deliberately trying to break your own game, on purpose, so you can find problems before anyone else does.

Professional game studios have entire teams dedicated to testing, called QA (Quality Assurance). As a solo developer, you get to be your own QA team!

Here's a simple testing checklist to run through with Alien Dodge:

- Does the ship stop exactly at both edges of the screen, instead of sliding off?
- Does every alien reset correctly to a new random position after falling off the bottom?
- Does a collision correctly remove exactly one life, not zero and not two?
- Does the game correctly display GAME OVER the moment lives reach 0, and stop responding to key presses afterward?
- Does the game correctly display YOU WIN! once the score target is reached?
- If you added a second alien or an asteroid, do all enemies fall and collide independently, without interfering with each other?

```
lives = 3

def take_damage():
    global lives
    lives -= 1
    print(f"Lives remaining: {lives}")

take_damage()
take_damage()
take_damage()
take_damage()
```

□ Here's What's Happening:

This is a tiny, isolated test instead of playing the whole game, we call `take_damage()` four times in a row to check exactly what happens once lives drops below zero.

Testing small pieces of your code by themselves, like this, is called unit testing, and it's a habit used by professional programmers everywhere.

Running this snippet would reveal a subtle bug: our lives variable can drop to -1, even though it should probably stop at 0. Finding a small bug like this in an isolated test is much easier than hunting for it inside a fast-moving game loop.

Here's one way to fix that exact bug, using an if statement to stop lives from dropping below zero:

```
def take_damage():
    global lives
    if lives > 0:
        lives -= 1
    print(f"Lives remaining: {lives}")
```

□ Here's What's Happening:

if `lives > 0`: only allows the `lives -= 1` line to run when there's still at least one life left to lose.

This means once lives reaches 0, calling `take_damage()` again simply does nothing further, rather than letting the count drop to -1 or lower.

This is a great example of how testing (finding the -1 bug) directly leads to a small, targeted fix, rather than a big rewrite.

Writing Checks With `assert`

Python has a built-in keyword, `assert`, that lets you write a quick, automatic check: "I expect this to be true stop and tell me immediately if it isn't."

```
def add_numbers(a, b):
    return a + b

assert add_numbers(2, 3) == 5
assert add_numbers(-1, 1) == 0
print("All tests passed!")
```

□ Here's What's Happening:

`assert add_numbers(2, 3) == 5` calls our function and checks whether the result really does equal 5. Since it does, Python quietly continues on to the next line.

`assert add_numbers(-1, 1) == 0` checks a second case, including a negative number, which is also correct.

If either assert line's condition had turned out to be False, Python would immediately stop the program and raise an AssertionError, pointing you straight at the exact line that failed.

`print("All tests passed!")` only runs if every assert line above it succeeded a simple, satisfying way to confirm your function behaves the way you expect.

□ **Tip:**

Writing a few assert lines for an important function, like one that calculates damage or checks a win condition, takes only a minute and can catch bugs instantly the next time you change that function.

□ **Mini Challenge: Break Your Own Game**

Play your game five times, trying a different "weird" thing each time mashing multiple movement keys at once, waiting without moving, pressing keys before the window fully loads.

Write down anything that looks or feels wrong, even if it's small.

Write two assert statements testing a small function from your own game, like a healing or damage function.

Improving UX

UX stands for User Experience how a game feels to actually play, not just whether it technically works. A game can be bug-free and still feel confusing, boring, or frustrating. Let's improve how Alien Dodge feels.

Here are a few beginner-friendly UX improvements you can make right now:

Instructions Before Playing

```
instructions = turtle.Turtle()
instructions.hideturtle()
instructions.penup()
instructions.color("white")
instructions.goto(-260, 0)
instructions.write("Use LEFT and RIGHT arrows to dodge!\nReach
500 points to win.", font=("Arial", 14, "normal"))
```

□ **Here's What's Happening:**

This uses the exact same "invisible writing turtle" pattern from Week 4's scoreboard, but this time to show the player instructions before the action starts.

The `\n` inside the string is a line-feed escape character the same one you learned about in Week 1 which starts a new line, letting us show two lines of instructions in a single `write()` call.

In a more advanced version, you could pause the game loop until the player presses a key to begin, so they always have time to read this message first.

Visual Feedback

Right now, getting hit just quietly reduces the life count. Let's make it feel more obvious.

```
if alien.distance(player) < 20:
    lives -= 1
    window.bgcolor("dark red")
    window.update()
    window.ontimer(lambda: window.bgcolor("black"), 100)
    alien.goto(random.randint(-280, 280), 280)
```

□ Here's What's Happening:

`window.bgcolor("dark red")` instantly flashes the background a warning color the moment a collision happens, giving the player clear visual feedback that they were hit. `window.ontimer(lambda: window.bgcolor("black"), 100)` schedules the background to flip back to black 100 milliseconds later. The `lambda` keyword here creates a tiny, unnamed, throwaway function just enough code to say "set the color back to black" without needing a whole separate `def` block for something this small.

This kind of small, satisfying feedback flashes, sounds, or animations that respond instantly to what the player does is a huge part of what makes games feel good to play, separate from whether they work correctly.

□ Tip:

Small UX touches like sound effects, screen shakes, or color flashes are often called "juice" by professional game developers. A little juice goes a long way toward making a simple game feel exciting.

Difficulty Curves

Another important UX idea is the difficulty curve how challenging a game feels over time. A game that's the exact same difficulty from start to finish often feels flat and boring. Let's make the alien speed slowly increase as the score grows.

```
fall_speed = 5

# Inside game_loop:
```

```
alien.sety(alien.ycor() - fall_speed)

if score % 100 == 0:
    fall_speed += 1
```

□ Here's What's Happening:

`fall_speed = 5` replaces the hard-coded number 5 we used before, storing it in its own variable so we can change it over time.

`score % 100 == 0` uses the modulo operator from Week 1 to check whether the score is an exact multiple of 100 in other words, whether the player has just crossed another hundred-point milestone.

Each time that happens, `fall_speed += 1` makes the alien fall slightly faster, gradually increasing the challenge as the player racks up more points.

This is a simple version of a difficulty curve: the game starts easy enough to learn, and gradually ramps up to stay interesting for a skilled player.

□ Mini Challenge: Add Your Own Juice

Add a different color flash for winning (like a green flash) versus losing.

Add an instructions screen using the pattern above, customized with your own wording.

Add the increasing `fall_speed` difficulty curve to your own game, and playtest whether it feels fair.

The Art of Debugging

We introduced debugging basics back in Week 1. Now that your programs are much bigger, let's build real debugging skills that will serve you for your entire coding journey.

When something goes wrong, resist the urge to guess randomly. Instead, follow this process:

51. Reproduce the bug: figure out the exact steps that make the problem happen every time.
52. Read the error message carefully, from the bottom up, and note the exact line number Python points to.
53. Isolate the problem: comment out or temporarily remove other code so you're only looking at the part that's broken.
54. Add temporary `print()` statements to check what your variables actually contain at different points in the code this is one of the simplest and most powerful debugging tools that exists.

55. Fix one thing at a time, then test again before making another change.

```
def game_loop():  
    print(f"DEBUG: alien y = {alien.ycor()}, lives = {lives}")  
    alien.sety(alien.ycor() - 5)  
    # ...rest of the function
```

□ Here's What's Happening:

Adding a `print()` line like this at the top of a function is often called a debug print or a print statement trace. It doesn't fix anything by itself it simply reveals what's really happening inside your program while it runs.

By watching these values scroll by in the Run window as the game plays, you can often immediately spot the moment a variable becomes something unexpected, which points you straight to the bug.

Once you've found and fixed the bug, remember to delete or comment out your debug print lines leaving them in would clutter your Run window during normal play.

□ Tip:

The prefix "DEBUG:" in the example above is a simple habit that makes it easy to spot (and later delete) your temporary debugging prints, separating them from any real messages your game is supposed to show the player.

Here are a few classic bugs to watch for in a turtle game like ours:

- Forgetting global inside a function that changes a variable created outside of it causing your changes to silently disappear.
- An off-by-one style mistake in a boundary check, letting the ship slide just slightly past the edge.
- Forgetting to call `scoreboard.clear()` before writing new text, causing old and new numbers to overlap into an unreadable mess.
- Forgetting `window.update()` somewhere, causing the screen to freeze even though the game logic is still secretly running underneath.

Let's walk through a full example of hunting down one specific bug, from symptom to fix. Imagine your scoreboard becomes an unreadable smear of overlapping numbers after a few seconds of play.

56. Reproduce it: you notice it always happens after the score updates a few times, not right away.
57. Since there's no error message (the game doesn't crash), you check your code for anything unusual around where the scoreboard text is written.
58. You spot that `scoreboard.write(...)` is being called every loop, but `scoreboard.clear()` is missing right before it.

59. You add the missing `scoreboard.clear()` line, directly above the `scoreboard.write(...)` line.
60. You re-run the game and confirm the scoreboard now updates cleanly instead of overlapping.

□ **Tip:**

Notice this process didn't involve guessing randomly at all it involved carefully observing what actually happened, forming a specific idea about the cause, and testing that idea directly. This approach works for bugs of any size, in any language.

□ **Mini Challenge: Bug Hunt**

Deliberately remove the word `global` from your `game_loop` function and run the game. Watch what breaks, and read any resulting error carefully.

Put `global` back, then deliberately remove `scoreboard.clear()` and observe what happens to the on-screen text.

Fix both issues, and write one sentence describing what you learned from watching each bug happen on purpose.

Presentation Day

You built a real game, from your very first `print("Hello, World!")` all the way to a fully playable, winnable game with scoring, lives, and collision detection. Presentation Day is your chance to show it off with pride.

Here's a simple format for presenting your game to family, friends, or classmates:

61. Name your game and give a one-sentence pitch: what is it, and what's the goal?
62. Demonstrate the controls live, and let your audience try playing it themselves.
63. Share one thing you're especially proud of in your code maybe the collision detection, the scoring system, or a UX touch you added.
64. Share one bug you found and fixed, and briefly explain how you tracked it down.
65. Share one idea for how you'd expand the game further if you kept working on it.

□ **Tip:**

Presenting your work out loud, even to just one person, is a skill in itself and it's exactly what real software developers do constantly, whether it's showing a manager a new feature or a friend a personal project.

Going Further: Saving a High Score to a File

Right now, your score disappears the moment the game window closes. Let's add one more real feature: saving the player's best score to a file, so it's remembered the next time the game runs.

```
def save_high_score(new_score):  
    with open("high_score.txt", "w") as file:  
        file.write(str(new_score))
```

□ Here's What's Happening:

`open("high_score.txt", "w")` opens (or creates, if it doesn't exist yet) a file named `high_score.txt` in "w" mode, which stands for write mode meaning we're about to replace whatever is currently in the file.

The `with` keyword automatically takes care of properly closing the file once we're done with it, even if something goes wrong it's the standard, safe way to work with files in Python.

`as file` gives us a variable, `file`, that represents the open file so we can work with it inside the indented block.

`file.write(str(new_score))` writes our score into the file. Since `write()` only accepts strings, we convert `new_score` to a string first using `str()`, just like we practiced with type casting back in Week 1.

Now let's read that saved score back, the next time the game starts:

```
def load_high_score():  
    try:  
        with open("high_score.txt", "r") as file:  
            return int(file.read())  
    except FileNotFoundError:  
        return 0
```

```
high_score = load_high_score()  
print(f"Current high score: {high_score}")
```

□ Here's What's Happening:

`open("high_score.txt", "r")` opens the file in "r" mode, standing for read mode, since this time we only want to look at what's already saved, not overwrite it.

`file.read()` reads the entire contents of the file back as a string, and `int(...)` converts it back into a real integer, since everything read from a file comes back as a string, just like `input()`.

The `try` and `except FileNotFoundError` block handles the very first time the game ever runs, when `high_score.txt` doesn't exist yet. Instead of crashing with an error, the `except` block catches that specific problem and safely returns 0 instead.

`high_score = load_high_score()` calls our function and stores whatever score it finds (or 0, if this is the first time playing).

Finally, here's how you'd tie both functions into your game's win/lose conditions, updating the saved file only when the player beats their previous best:

```
if lives <= 0:
    if score > high_score:
        save_high_score(score)
        scoreboard.write("NEW HIGH SCORE!", font=("Arial", 20,
"bold"))
        scoreboard.goto(-70, 0)
        scoreboard.write("GAME OVER", font=("Arial", 28, "bold"))
    return
```

□ **Here's What's Happening:**

`if score > high_score:` compares the score from this specific playthrough against the best score ever saved to the file.

If the player beat their own record, `save_high_score(score)` writes the new, higher number into `high_score.txt`, permanently replacing the old saved value.

The rest of the block behaves exactly like our earlier GAME OVER logic, just with an extra congratulatory message when a new record is set.

□ **Tip:**

`try` and `except` are Python's way of gracefully handling errors you expect might happen, instead of letting your whole program crash. We used it here for a missing file, but the exact same idea works for many other predictable problems.

Ideas for Going Even Further

If you want to keep building after this guide ends, here are a few more beginner-friendly directions to explore:

- Add power-ups: a special item the player can catch that temporarily gives an extra life or a speed boost.
- Add a second, harder difficulty mode the player can choose before the game begins.
- Explore the `pygame` library, a more advanced game-making toolkit that unlocks smoother animation, sound effects, and sprite images.

Your Complete Alien Dodge Code

Here is the entire game, combining every piece we built across Week 4 and Week 5, all in one place for you to reference.

```
import turtle
import random

window = turtle.Screen()
window.title("Alien Dodge")
window.bgcolor("black")
window.setup(width=600, height=600)
window.tracer(0)

player = turtle.Turtle()
player.shape("triangle")
player.color("cyan")
player.penup()
player.goto(0, -250)

alien = turtle.Turtle()
alien.shape("circle")
alien.color("red")
alien.penup()
alien.goto(random.randint(-280, 280), 280)

scoreboard = turtle.Turtle()
scoreboard.hideturtle()
scoreboard.penup()
scoreboard.color("white")
scoreboard.goto(-280, 260)

score = 0
lives = 3
fall_speed = 5

def load_high_score():
    try:
        with open("high_score.txt", "r") as file:
            return int(file.read())
    except FileNotFoundError:
        return 0
```

```

def save_high_score(new_score):
    with open("high_score.txt", "w") as file:
        file.write(str(new_score))

high_score = load_high_score()

def move_left():
    new_x = player.xcor() - 20
    if new_x < -280:
        new_x = -280
    player.setx(new_x)

def move_right():
    new_x = player.xcor() + 20
    if new_x > 280:
        new_x = 280
    player.setx(new_x)

window.listen()
window.onkeypress(move_left, "Left")
window.onkeypress(move_right, "Right")

def game_loop():
    global score, lives, fall_speed

    alien.sety(alien.ycor() - fall_speed)
    score += 1

    if score % 100 == 0:
        fall_speed += 1

    if alien.ycor() < -300:
        alien.goto(random.randint(-280, 280), 280)

    if alien.distance(player) < 20:

```

```

        lives -= 1
        window.bgcolor("dark red")
        window.ontimer(lambda: window.bgcolor("black"), 100)
        alien.goto(random.randint(-280, 280), 280)

    scoreboard.clear()
    scoreboard.write(f"Score: {score}  Lives: {lives}  Best:
{high_score}", font=("Arial", 16, "normal"))

    if lives <= 0:
        if score > high_score:
            save_high_score(score)
            scoreboard.write("NEW HIGH SCORE!", font=("Arial",
20, "bold"))
            scoreboard.goto(-70, 0)
            scoreboard.write("GAME OVER", font=("Arial", 28,
"bold"))
            return

        if score >= 500:
            scoreboard.goto(-100, 0)
            scoreboard.write("YOU WIN!", font=("Arial", 28,
"bold"))
            return

    window.update()
    window.ontimer(game_loop, 33)

game_loop()
window.mainloop()

```

□ Here's What's Happening:

This full listing is exactly the game we built piece by piece: setup and game objects at the top, our high-score file functions, our two keyboard functions and their event handling, and finally the `game_loop` function that ties movement, randomness, collision detection, scoring, a difficulty curve, visual feedback, high scores, and win/lose conditions together.

Reading a full program like this from top to bottom, and being able to explain what every single line does, is one of the clearest signs that you've truly learned to code not just copy code, but understand it.

Use this listing as your personal reference. Compare it closely to your own file if anything in your game isn't behaving quite right often, the fix is a single missing or misplaced line.

Writing Code for Other People to Read

Right now, you're the only person who's read your game's code but real projects are almost always read by other people too, including a future version of yourself who has forgotten exactly how a tricky section works. A few simple habits make a huge difference.

- Use clear, descriptive variable and function names, like `fall_speed` instead of `fs`, so the code explains itself.
- Add a short comment above any section of code that isn't immediately obvious, especially clever tricks like the lambda color-flash from earlier this week.
- Keep related code grouped together all your game object setup in one place, all your functions together, and the game loop clearly separated at the bottom.
- Remove old debug prints and commented-out code you're no longer using before sharing or presenting your project.

```
# Handles player movement and keeps the ship within the screen
bounds
def move_left():
    new_x = player.xcor() - 20
    if new_x < -280: # left edge of the 600-pixel-wide window
        new_x = -280
    player.setx(new_x)
```

□ Here's What's Happening:

The comment above the function briefly explains its overall purpose useful for anyone skimming through many functions quickly.

The comment beside the `if` line explains why `-280` specifically was chosen, connecting it back to the window's size a detail that might not be obvious to someone reading only this line.

Notice these comments explain the why, not just the what line-by-line commentary in this style is far more valuable than comments that simply restate what the code already says clearly.

□ Tip:

A good rule of thumb: if you had to think for more than a few seconds to understand what a piece of code does, a short comment explaining it will save the next reader (including future you) that same time, every single time they read it.

A Quick Note on Saving Your Work

As your projects grow, it becomes more important to keep track of your changes over time. Many professional programmers use a tool called version control (the most common one is called Git) to save snapshots of their project as it develops, so they can always look back at or even restore an earlier working version.

You don't need Git to finish this guide, but here are two simple, low-tech habits that give you similar protection:

- Before making a big change, copy your whole project folder and rename the copy with today's date, so you always have a safe, working backup to fall back on.
- Keep a personal changelog a simple text file where you jot down a one-line note each time you add a feature or fix a bug, like "Added high score saving" or "Fixed scoreboard overlap bug."

Week 5 Recap

- Testing means deliberately trying to break your own game to find problems before players do.
- Isolated, small tests of just one function (like `take_damage()`) are called unit tests; `assert` lets you write these checks directly in code.
- UX (User Experience) is about how a game feels to play, including instructions, visual feedback, and a difficulty curve.
- Debugging is a repeatable process: reproduce, read the error, isolate, add debug prints, and fix one thing at a time.
- `open()`, `with`, `.write()`, and `.read()` let a program save and load information in a file, even after the program closes.
- `try` and `except` handle predictable errors gracefully, like a missing file, instead of crashing the whole program.
- Presenting your work clearly the goal, the controls, a proud moment, a fixed bug, and a future idea is a valuable skill on its own.

Check Your Understanding

66. What is the difference between testing and debugging?
67. Give one example each of a UX improvement and a bug fix, and explain why they're different kinds of changes.
68. Why is it helpful to add a temporary "DEBUG:" print statement instead of just guessing what a variable might contain?

69. What does "w" mode do differently from "r" mode when opening a file?
70. Why do we wrap `load_high_score()` in a try/except block instead of just calling `open()` directly?
71. Why is presenting your finished project to someone else a valuable step, even after the code itself is done?

Full Course Glossary

Here is every key term from all five weeks, gathered in one place as a lasting reference you can flip back to any time you need a quick reminder.

- **Algorithm** a clear, ordered sequence of steps that solves a problem, whether or not a computer is involved.
- **Argument** the actual value supplied to a function when it's called, matched up with its parameters.
- **Assignment operator (=)** stores a value inside a variable.
- **Boolean** a data type that is only ever True or False.
- **Bug** a mistake in code that causes incorrect or crashing behavior; debugging is the process of finding and fixing bugs.
- **Collision detection** checking whether two game objects have touched, often using a distance calculation.
- **Comment (#)** a note for human readers that Python ignores completely.
- **Comparison operator** a symbol like `==`, `!=`, `>`, or `<` that compares two values and produces a boolean.
- **Concatenation** joining strings together, often using the `+` operator.
- **Data type** the kind of information a value represents: string, integer, float, or boolean.
- **Dictionary** a data structure that stores key-value pairs, looked up by a meaningful key.
- **Event handling** connecting a specific action, like a key press, to a function that runs in response.
- **f-string** a string prefixed with `f` that lets you insert variables and expressions directly using curly braces.
- **Float** a number with a decimal point.
- **Function** a reusable, named block of code, created with `def`.
- **Game loop** the repeating cycle of checking input, updating the game, and redrawing the screen.
- **Global** a keyword that lets a function modify a variable defined outside of it.

- **Index** a number representing a position within a list, starting at 0.
- **Infinite loop** a loop whose condition never becomes False, causing it to repeat forever.
- **Input()** a function that pauses a program to ask the player for typed information.
- **Integer** a whole number, with no decimal point.
- **List** an ordered, changeable collection of values.
- **Logical operator** and, or, or not, used to combine or flip boolean conditions.
- **Module** a bundle of pre-written tools, brought into a program using import.
- **Nested** a structure (like an if or a loop) placed inside another one of the same kind.
- **Parameter** a placeholder name listed in a function's definition, filled in by an argument later.
- **Print()** a function that displays information on the screen.
- **Random module** Python's built-in toolkit for generating unpredictable values.
- **Return** a keyword that hands a value back out of a function.
- **String** text, written between quotation marks.
- **Type casting** deliberately converting a value from one data type to another, using int(), float(), or str().
- **Tuple** an ordered collection of values that cannot be changed once created.
- **Variable** a named storage location that holds a value, which can change over time.

Congratulations, and great job finishing this journey! You started five weeks ago not knowing what a variable was, and now you understand variables, data types, conditionals, loops, randomness, functions, lists, dictionaries, graphics, coordinates, game loops, collision detection, scoring systems, saved high scores, testing, debugging, and UX. That is a real, professional-level foundation in programming keep building, keep experimenting, and keep having fun with code.